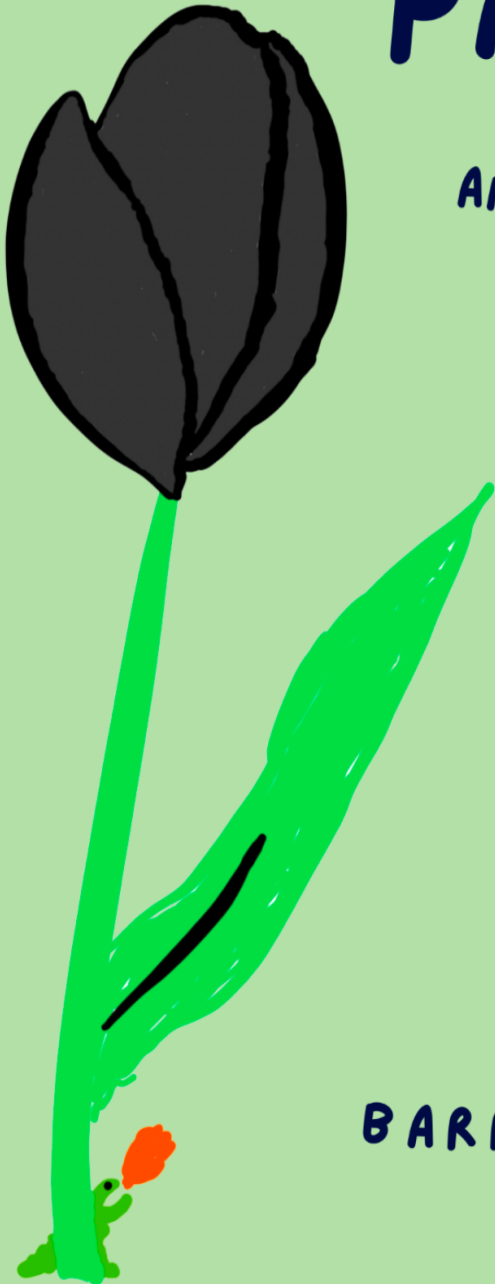


THE ARCHITECT'S PARADOX

UNCERTAINTY
AND THE PHILOSOPHY
OF SOFTWARE
ARCHITECTURE



BARRY M O'REILLY

The Architect's Paradox

Uncertainty and the Philosophy of Software Architecture

Barry O'Reilly

This book is available at <http://leanpub.com/architectsparadox>

This version was published on 2025-01-31



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2025 Barry O'Reilly

*For Tanya, Alexander, and Freya, whose default philosophy has
always been chaos.*

Contents

Introduction	1
How this book is structured	6
SECTION I: Computing and the Philosophies of Stillness	9
Walk 1: What is Architecture?	10
Starting with Plato	12
Architecture in Theory	15
Verifying Conceptual Structures	19
Walk 2: Certainty and Uncertainty.	20
Reductionism	23
Walk 3: Determinism, Reductionism, Logic and the Birth of Computing	27
The Birth of Computing	28
Requirements “engineering”	33
Coping with change	36
Walk 4: Chaos and the Philosophies of Stillness.	38
Parmenides, Plato and our obsession with stillness	42
The Accidental Philosophy of Software Architecture . . .	46
Walk 5: Essentialism.	48
Platonic Forms and Abstraction	50
How this turns up in software.	53

CONTENTS

Walk 6: Causality, Mechanism, and Determinism	57
How does this apply to architecture?	59
Essentialism, Determinism, and Causality	60
What Does This Mean for Software Architecture?	64
Determinism, Essentialism, and the Age of the Machine: Cybernetics	66
Walk 7: Logical Structuralism	70
What Does This Mean for Software Architecture?	72
Economics, Trade-offs & Neoliberalism	75
Modeling Logical Structuralism	77
Baudrillard	78
Summary: The Default Philosophy of Architecture	79
Walk 8: Skepticism, Rationalism, and Empiricism	82
Skepticism in Software	87
SECTION II: The World That Refuses to Be Still	88
Interlude	89
Architecture & Metaphysics	90
Walk 9: Being and Becoming.	92
Walk 10: Kant, Hegel, and How We Think About Things	95
Relative a priori	102
Hegel brings back becoming	105
Walk 11: Reversing Platonism and the Crisis of Meaning	108
Analytic vs Continental Philosophy	114
SECTION III: The Philosophies of Movement	115
Walk 12: Philosophies of Movement	116
Process Philosophy	117
Post-structuralism	121
Pragmatism	124

Induction, Deduction, and Abduction.	126
Summary	127
Evolution	129
Walk 13: Emergentism	132
Emergentism	133
Emergentism as Science	139
Walk 14: Complexity Science	140
Complexity and Model Drift	146
A Better Approach to Complexity.	147
The Deleuzian Walk	149
Walk 15: Residuality Theory	153
Architecture as Science	159
What do we do with it?	160
How should an architect think	161
Walk 16: Conclusion.	162
Acknowledgements	165

Introduction

This book was an accident. It started as a chapter on philosophy for my first book, *Residues: Time, Change and Uncertainty in Software Architecture*¹. Though I wasn't entirely sure that it would be a good fit for the style of the book, I resolved to write it anyway. After only a few hours, I realized that the ideas I was developing were quickly growing beyond the scope of the book. But I was enjoying myself, so I kept going, and before long, I had written an entirely new book, longer than the first one.

You might be wondering why a software engineer like me would bother taking an interest in philosophy in the first place. I have already mentioned that this book happened by accident. In many ways it was sparked by a chance conversation with my wife, a scholarly lady who owns many books. I was explaining why the tendencies of software architects seem to be constrained by their training, that the world is not always as it seems to them, that their preconceived model often gets in the way, and they have no way to express these difficulties. In response, she handed me a book, *The Parasite*, by Michel Serres. While I completely failed to grasp the main arguments, its philosophical prose sent me into a spiral that I can only describe as “*A software engineer getting lost in the wrong part of the library*”. In the years that followed I engaged on a deep level with philosophical ideas, hundreds of books, essays, discussions, and total and utter confusion. This journey was tough, but it did something code hadn't been able to do for a long time; it entertained and inspired me, and it gave me hope that there was something more, something not yet discovered, that could make the designing of software systems easier.

I wrote *The Architect's Paradox* as a follow up to *Residues*, for

¹O'Reilly, B.M. (2024)*Residues: Time, Change and Uncertainty in software Architecture*, LeanPub <https://leanpub.com/residuality>

those who want to dig deeper into the philosophical reasons why traditional architecture is doomed to fail and requires a different approach. The book has two goals in mind. It is my hope that this book will also serve as a fun introduction to philosophy for software engineers who don't really get a chance to study these things at university. This new book retraces the steps of my difficult but rewarding philosophical journey that began when I realized that there was no school of thought within software engineering, computer science, or even the complexity sciences that was going to solve the problems inherent in software architecture. This revelation forced me to return to first principles, and take a peek behind the curtain, to see what was really going on.

For the past 10 years, my work has involved applying the lessons of Complexity Science to the practice of Software Engineering. As seen in my previous book, *Residues*, this has had some pretty dramatic effects! My original suspicions that traditional architectural techniques weren't really working as they were taught, coupled with my study of the complexity sciences, have led me down some unexpected paths which I will briefly retrace here.

After I realized that the view of the world we acquire through our computer science educations failed to translate in the real world, I knew there was a real need to look elsewhere. Interestingly, I found that many senior architects had already come to this conclusion and had become comfortable with a constant level of uncertainty. This impelled me to explore exactly what this uncertainty entailed.

Those trained in the mathematical disciplines (most of us software architects that is) will first turn to statistics and perhaps Bayesian thinking to rescue their ideas from uncertainty, but again, this has limited use in the real world of software engineering. The gap between the scientist's need to say only what is observably true, and the need to speculate in order to get anything done in the real world of uncertain futures, leaves the architect in a twilight zone in which they lie to themselves and to others about how their work is

being carried out. Look at Parnas' 1986 paper² that recommends simply pretending that there is some kind of rational structure behind our approaches even when there clearly isn't. The failure of architecture does not boil down to a lack of competence, it is a full-blown identity crisis.

To investigate this further I turned to the field of the complexity sciences. This offered few immediate solutions, but it was thoroughly entertaining. The effect of this was that I ended up using the complexity sciences as something like a theoretical can opener, tearing the lid off the suppressed identity crisis of software architecture.

The first thing that became apparent as I presented the new way of thinking developed in my first book was the fear of questioning established ideas in the industry. For many people, new ideas spark curiosity, but for some, they spark fear and uncertainty, especially when a pre-existing and entrenched set of beliefs seemingly already meets their needs. I began to dig deeper to try and understand why there was so much resistance and why so many were intent on stubbornly sticking to frameworks that didn't work and offered no evidence beyond the anecdotal. Complexity science was able to split open the dogma at the heart of many ideas in software engineering and reveal some fascinating philosophical splits. But it simply wasn't enough for me to debate at the level of frameworks and tools: the positions held by many software engineers were so philosophically distant from one another, and the debaters were so deeply unaware of this, that there could never be any resolution. In software terms, we were stuck in an infinite loop.

This is how I ended up in "the wrong section of the library" where I spent several years reading as much as I could until a narrative began to unfold. My subsequent articles, published in philosophy journals, caused some concern among my PhD supervisors who watched in horror as I hurtled toward post-modernism and began

²Parnas, D. L., & Clements, P. C. (1986). A rational design process: How and why to fake it. *IEEE transactions on software engineering*, (2), 251-257.

questioning the very underpinnings of traditional software engineering.

The results of this research were my favorite part of the journey toward my PhD. At a conference in Stockholm in 2021 I spoke publicly for the first time about the philosophy of software architecture. While the computer science professor who appeared on stage after me was a little shocked by my attack on the pervasive positivism of computer science, the general response from the audience was one of surprise and curiosity. In the spring of 2023, I held the same talk at a developer conference and it went down equally well there. In the autumn of 2023, I sold out two short, online courses on the subject and, by then, it had started to feel like this wasn't simply a niche hobby, but perhaps something of greater interest to the wider industry.

For those of you seeking a deeper understanding of the ideas in my first book, how they came to be and how they are related to each other, you will find it here. For those of you in the software industry simply seeking to understand more about the relationship between philosophy and software and to be gently introduced to a subject alien to most of us, this should be made easy by the constant references to software.

If you are a philosopher, this book might help you to understand the weird and wonderful world of software engineering. Here, the software engineer will find themselves at odds with the academic philosopher. Trying to explain exactly what a long dead philosopher meant by a phrase in their time and context from the perspective of our time and context is immensely difficult, if not impossible. If a software engineer should misunderstand a particular concept according to current academic consensus, and yet produce a better software system because of the questions that misunderstanding leads to, then the engagement with philosophy has been fruitful. One of the biggest problems in moving a logically rigid programmer to the thinking of an architect is freeing them from the need to be right all the time, to take a risk and think a

new thought. These brave leaps will be encouraged here, at the cost of historical accuracy or alignment with current or imagined academic consensus. Programmers already live in a self made intellectual prison, we don't need another one.

This book will reveal that the rigid logical thinking that created computing creates the conditions for architectural failure, *the architect's paradox*. In the same way, the gatekeeping that some practice within philosophy and expected adherence to contemporary interpretations of thinkers can be a barrier to using philosophy creatively for something useful. Making students read and recite Knuth before being allowed to write code would theoretically produce better code in some instances, but it would also reduce the number of programmers by orders of magnitude. Philosophers used to careful contemplation and cautious arguments might find the iterative, pragmatic approaches to the thinking of software engineers brash and uncouth, but this is the price of transdisciplinary work.

The history of philosophy is an immensely interesting and captivating subject. However, the average software engineer is an immensely pragmatic person who wants to build something. In an industry that demands constant action, philosophy is regarded with suspicion, as navel gazing. The purpose of introducing these ideas is to force new thinking, to force something to happen. Deep investigations of what a particular thinker thought at a particular time are interesting, but not the purpose of this work. Challenging our own thinking, and finding new ways of considering the world, can be done simply by reflecting on the ideas in front of us, and as such many of the ideas presented here are not presented in depth, with all the many arguments and counter arguments, as that would alienate the intended technical audience and reduce the chances of any active attempt to think anew. I am interested in using philosophy to produce something new, not to analyze something old.

How this book is structured

Every chapter in the book is designed as a “walk” around the problem of architecture and how we think about software design conceptually. This walk might be triggered by a particular idea, such as uncertainty or essentialism, or it might be a particular thinker like Kant or Hegel. Each walk should force you to take a step back and think about how you describe, think about, or become aware of the business context around you when you are creating software structures, and the point is to force you to think independently about your own practice before we get to Section III and present some alternative ways to see the world. Each walk will add new layers to your thought and introduce new philosophical concepts and thinkers, or sometimes revisit old ideas with a new lens based on earlier walks. By the time you get to Walk 14 you’ll understand how this concept of walking applies to architecture itself - and you’ll have a pretty good grasp of the major ideas and thinkers in Western philosophy and how the ideas can be applied to designing software systems.

If you are a fan of residuality and want to get straight to the point then read Walks 1-4 and then skip to Walk 13. Then you can go back and read through the other walks at your leisure. If you like philosophy and want the whole deep journey from Heraclitus to Residuality start at the start and read the whole book. If a walk feels too heavy then just skip to the next one and come back to it later. This is a very different book than what developers are used to. Reading it in one sitting will be a challenge. Reading it a second time will reveal new angles and thoughts, every time you repeat your reading of the book you will notice different things and think different thoughts.

It might take several reads to grasp everything. Don’t try to memorize or study the text - there is no exam - the book is designed to be walked many times. It’s important to read the book with an

aim to triggering your own internal reflection about your practice of architecture, not as an exercise in learning facts. The aim is that the book will reflect your thinking back at you, acting as a mirror, allowing you to better understand your own process. If you try to analyze the book and learn the content the same way you learn a programming language you will miss the point. Later in the book I will talk about Henri Bergson's reflection that if you focus on identifying the notes you won't hear the tune, and this would be a perfect example of the architect's paradox in action if you try to learn all the ideas at once.

Some of these ideas might seem challenging to a software developer, especially if you haven't designed any large scale enterprise systems yet in your career. It might seem like even daring to question the established opinions and ideas risks destroying the only things you have to lean on. The book is not attempting anything so dramatic. Instead, view this as the start of a journey that allows you to begin to question the world around you in a new way. This will inevitably happen as you work with bigger and tougher challenges, and this book should give you some tools to think through that when the day comes.

You might find the book overwhelming at points. In this book I will describe the ideas of a large number of thinkers. You will probably struggle to remember their names, the names of the different schools of thought, the abstract concepts each introduces. If you have a background in programming your instincts will probably tell you not to move forward until you have grasped and understood all the details. Relax! I was once told that studying philosophy is like attending dinner parties with your in-laws. The first time you attend, you don't know who anyone is, you don't know who is related to who, and you forget everyone's name immediately as you are introduced to them. After a few parties you start to get a glimpse of who's who, who is interesting to talk to, what they think about the world. You also learn who to avoid! It takes many years before you understand all the details of past dramas and

relationships, before you understand the in-jokes and the dynamics. In my experience of learning philosophy there is absolutely no point in cramming these details as if there were a test, just relax and enjoy it.

I hope you enjoy this little excursion through the history of philosophy and that you will see the many links to how we think as software architects, and most importantly, the many potential ways we *could* think in the future.

SECTION I: Computing and the Philosophies of Stillness

Walk 1: What is Architecture?

This book will advance an argument that I call *the architect's paradox*. The study of logic that allowed us to create computing machines has created a residual, unconscious philosophy that has arrested our development such that architecting software structures has become very difficult and has even arrived at a state of crisis. I will start this journey by trying to define what architecture is.

Software architecture is not a clearly defined term and attempts to create a body of knowledge are usually overly verbose, unclear, unfinished, and decay quickly. Despite existing as a concept in the software engineering literature for decades, there is no real agreement on the definition of software architecture, what it should do, or what it is for. There are very many definitions. Some are prescriptive and detailed; others are blanket statements. “It’s the significant decisions” says Grady Booch. “It’s the stuff that’s hard to change later” says Martin Fowler, and they’re both right. The debate has been so meandering and so diffuse that there are very many in our industry who no longer want anything to do with the idea. In the eyes of many in our industry, architecture has failed.

This book will attempt to dig into these questions in a slightly different way. Instead of answering the question directly, or simply stating new ways of defining architecture, this book will analyze the state of architecture through the lens of Western philosophy, a journey revealing that most of our problems, and all of our solutions, are not new ground, but have been walked many times before.

As such, I will try to answer the following questions:

Why is this so hard?

Why are we like this?

What can we do about it?

Starting with Plato

One of the most important thinkers in the history of Western thought is Plato. He was born and lived in Ancient Greece, around 428-348 BC. He created a system of ideas that would make their way into the Roman Empire, and into the Catholic Church through a successor called Plotinus, and continued to influence European thought all the way to the present day. One of the philosophers I will talk about in a later chapter, Alfred North Whitehead, once famously referred to all of Western philosophy as “footnotes to Plato”, so great was the influence of Plato’s thought. As I will show in later chapters, Plato’s thinking is always present in much of the thinking around software architecture, albeit unconsciously.

Most of Plato’s works were written as dialogues, that is as conversations, in a style known as *dialectic* — a series of arguments. One of the heroes of these dialogues is a character called Socrates, based on a real person and philosopher known to Plato, but who never wrote down his own thoughts, who interrogates people by asking a series of questions, very often revealing their own ignorance to them. This is known today as the *Socratic Method*, and is not a particularly great way to make friends.

In one of these dialogues, Plato’s *Meno*, Socrates asks Meno to define virtue. His answer is a meandering set of *examples* of virtue. Socrates is not satisfied with this answer, he wants to know what virtue is, at its core. He is seeking the *essence* of virtue. The dialog does not arrive at an answer, ending in *aporia*—a kind of confusion, an absence of conclusion. In the end, it doesn’t seem possible to define virtue. This could nonetheless be considered a step forward, since now they know what they don’t know. The same thing happens in a number of Plato’s dialogues—for instance, in *Theaetetus*, where Socrates wants Theaetetus to define knowledge, and similar ideas are shown in *Gorgias*. Many of the Socratic dialogues are based around this kind of “What is it?” questioning, and many of them end in the same *aporia*. Defining things is hard.

Defining software architecture feels quite similar.

This book will chart a journey from Socrates' need to distinctly define concepts like virtue, and Plato's failure to do so, to a modern school of philosophy called emergentism—which turns out to be a way of thinking that makes software architecture much easier to reason about. I will argue that Western philosophy has followed an eerily similar path to that of software development, and therefore contains some clues about what might come next for software architecture. Before we get there, let's imagine a modern day *Meno*:

Socrates: What is Architecture? Can it be taught?

Meno: It's Zachmann¹ and TOGAF². It's modeling and frameworks.

(Meno simply lists examples of architecture, but fails to define what it is)

Theaetetus: No, no, it's alignment with the business.

(Theaetetus is a little bit cleverer, and attempts to identify a unifying concept. Plato tells us already that Theaetetus is cleverer because he's a mathematician)

Gorgias: It's "Technology Strategy for the Business". It's an *art* and a *science*. *Smug knowing look*

(Gorgias is Meno's teacher, and a sophist, a fancy Greek word for a bullshitter, who emphasizes giving answers that sound good, rather than knowledge. Gorgias has a framework to sell us)

Socrates: How come this poor, uneducated programmer can do it without your fancy training?

(In the *Meno*, Socrates shows that an uneducated slave has some intuitive knowledge of geometry, suggesting something innate that can be brought to the surface with the right guidance)

¹Pereira, C. M., & Sousa, P. (2004, March). A method to define an Enterprise Architecture using the Zachman Framework. In *Proceedings of the 2004 ACM symposium on Applied computing* (pp. 1366-1371).

²Haren, V. (2011). *TOGAF Version 9.1*. Van Haren Publishing.

The bad guys in the *Meno*, and the *Theaetetus*, are the *sophists*, like Gorgias and Protagoras—both of whom get their own dialogues. Sophists are those who excel at making arguments, bamboozling people, based on rhetoric, weak premises, and hand waving. Gorgias describes how powerful it is to be able to win over experts in their fields, without actually being an expert. This is exactly what software architecture needs to stop doing if it wants to be brought in from the cold. The aporia that arises when trying to define architecture creates a multitude of possibilities, and some of them are very weak while sounding good. This aporia is a breeding ground for sophistry.

Plato's *Gorgias* explores these themes, expounding on the fact that rhetoric is not knowledge, that it produces *belief without knowledge*. Rhetoric has a powerful place in a software engineering industry that does not stand on secure, established scientific foundations. Rhetoric is pervasive, it leads to dogma, and stops us from learning as a profession. In working with software architects and navigating the many and varied frameworks, belief without knowledge is a common feature.

Trying to define architecture is very much like Meno's attempt to define virtue, or Theaetetus' attempt to define knowledge, it ends in aporia, or confusion. Despite this aporia, there are very many definitions of architecture, often put forth with gusto and confidence.

Architecture in Theory

In 2008 a 14-year-old boy called Taylor Wilson surprised his local town by building a nuclear reactor in his garage. He did this with the help of some Physics textbooks he borrowed from his father. When theory is so close to reality as to allow a 14-year-old to endanger an entire town, then we can say that theory and practice are pretty close to each other.

This is a book about philosophy and one of the tools philosophers use is the thought experiment. So, as a first thought experiment, imagine that Taylor had taken a copy of any enterprise architecture framework into that garage. What would he have built? At best, I think, a small fire. On the positive side, perhaps it would have inspired him to find normal teenage hobbies.

This leads us to ask what the theory behind software architecture entails. There are a lot of books and ideas and debates, and no shortage of volunteers to tell us how we should be doing things, but in terms of theory, what do we have to lean on? There is received wisdom on things like patterns and practices that are popular. Of those practices, where is the scientific evidence that they work? Very often these popular ideas and movements are based on a ‘works on my machine’ kind of thinking. The result is an industry led by charisma and rhetoric, by cool stories and affable gurus, rather than theory and replication—*belief without knowledge*. This has reached the point where daring to ask hard questions is almost forbidden, as it might reveal that the emperor might not be fully dressed. Defining architecture is fraught with risks.

Software architecture is concerned with the structure of software programs. Structures are important because they are seen as the primary barrier protecting software against change in the business context. The business context is how we describe the organization, its business model, its market and competitors, its culture, its history, its employees and customers. Protecting the

software structure against change in this business context became a focal point when Dijkstra's GOTO Considered Harmful (1968)³ argued that software was extremely vulnerable to changes in its environment due to its brittle structure. The NATO conferences held in Garmisch, Germany, that same year decried the inability to deliver software in any reliable way. Starting at the NATO conferences there was an attempt to define software as an engineering discipline, which led to software importing all manner of tools from other engineering disciplines.

Dijkstra's GOTO article triggered the move to structured programming, which had already been presented two years earlier in Böhm and Jacopini's blocks⁴ (1966) that represent the first understanding of the role of modular structures in software, Kay's object orientation (1966)⁵, Parnas' information hiding (1972)⁶. Dijkstra's writing and speeches on abstraction (1972)⁷ and separation of concerns (1974)⁸ also influenced ideas about how to define software structures. In 1974, Fred Brooks declared in *The Mythical Man Month*⁹ that conceptual integrity is the key to software success, and in the same book, he separated the idea of the software and its concepts from the implementation. In this book, when I refer to the architecture, I similarly refer to the idea, the conceptual structure of the code, not the implementation, and treat these two things as separate skills.

How do we arrive at this conceptual structure? The architect must engage with the real world problem and decide what the

³Dijkstra, E. W. (1968). Letters to the editor: go to statement considered harmful. *Communications of the ACM*, 11(3), 147-148.

⁴Böhm, C., & Jacopini, G. (1966). Flow diagrams, turing machines and languages with only two formation rules. *Communications of the ACM*, 9(5), 366-371.

⁵Kay, A. C. (1996). The early history of Smalltalk. In *History of programming languages---II* (pp. 511-598).

⁶Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053-1058.

⁷Dijkstra, E. W. (1972). The humble programmer. *Communications of the ACM*, 15(10), 859-866.

⁸Dijkstra, E. W., & Dijkstra, E. W. (1982). On the role of scientific thought. *Selected writings on computing: a personal perspective*, 60-66.

⁹Brooks, F. P. (1974). The mythical man-month. *Datamation*, 20(12), 44-52.

constituent blocks of the software application should be. There have been many approaches described, but most involve similar steps to the following list:

- Identify the real world ‘things’ or entities involved in the problem
 - Like customer, order, etc.
- Place these things into categories
- Describe the properties of these things
- Describe the relationship of these things to each other
- Understand the rules for how they interact with each other
- Understand the causes and effects between these things
- Describe all of this as a conceptual structure
- Map that conceptual structure to software structures
- Map that software structure to an application running on hardware that meets the right levels of quality

This leads to an architecture. The way that this is done varies greatly between architects and organizations and is never done the same way twice. This book will force the question: Why do we do things this way? Why do we even think this is possible?

One of the key uses of philosophy is to force us to question our assumptions. This book will take you on a journey that will make you realize that the simple process described above is rife with assumptions, and that those assumptions have their roots in philosophical ideas that stretch back over two and a half millennia.

Funnily enough, creating a conceptual model involves the creation of concepts. What exactly is a concept, then? It turns out that there are many ways to answer this question, each answer revealing what we believe about the structure of the mind, about how we think about the universe. In creating a conceptual model, the architect

unwittingly makes a number of philosophical statements, many of which usually contradict each other.

For example, we can have what is called the *representational theory of the mind*, where thought creates representations of reality, and these representations have internal structure in our minds. This internal representation can take the form of images, or it can be language. Some believe that this internal representation is faithful to the real world, some believe that it is not at all faithful, and some even think that it *is* the real world! As architects we are used to the idea of concepts as mental representations—as sketches and models. This concept creation is, for most software architects, subconscious and unexplored. This entire book will focus on how and why we create these concepts, what lies at the base of our concept creation, and what might lead us astray.

Sometimes, we produce these conceptual structures in a certain way because the technologies we use demand it from us. Storing data in a structured way forces us to ask what structure the data can take, which leads to questions about entities, relations, and properties. Early computers had limited storage and computing power which led to an instinctive attempt to reduce the usage of these things and optimize our conceptual models for the technology. This led to a blurring of the lines between the problem and the technical solution. Many of the reflexes developed over the preceding decades are difficult to trace in this regard, often a mixture of the need for technical optimization and sufficient conceptual flexibility.

Verifying Conceptual Structures

Our first instinct, when trying to show that a program would execute correctly, was to use mathematics, another program, to prove that it was logically sound. However, Turing’s “On Computable Numbers”¹⁰, and the discovery of the halting problem (1936) showed that programs could not be completely verified before they execute. Hoare Logic (1969) then showed how software could instead be verified, although never completely, through empirical testing. The Scottish philosopher David Hume defined what is now known as Hume’s fork, separating all knowledge into either analytical statements verified by logic, or synthetic ideas verified by experience. All else, said Hume, was sophistry and illusion and should be set alight (you will find plenty of this Celtic tendency toward drama in the proceeding pages). The conceptual structures of software architecture are stuck between the prongs of this fork. Due to the halting problem, architecture cannot be verified analytically to be correct at inception, and by the time empirical testing can be run on actual code, the architecture is already set and too difficult to change. This places conceptual architecture in a strange world. We know it needs to be verified, but we don’t really know where, when, how, or if, we can show it to be correct. There are many software architects who have succeeded in finding some kind of balance here, producing designs prior to empirical verification of code that succeed in helping the software avoid the dangers of change in the business environment. This ability is often reported as “gut feeling”. It is this ability, this seemingly magical intuition that I have spent the last decade studying, and in this book, I will show how important it is that we gain a philosophical understanding of what we do, in order to quickly develop that ability and teach it.

¹⁰Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *J. of Math*, 58(345-363), 5.

Walk 2: Certainty and Uncertainty.

One of the key lessons that you learn as a software architect is that there is an enormous gap between what is certain and understandable in the software context, and what is uncertain and unmappable in the business context. As undergraduates the unpredictable, dynamic world of the business outside of software is not revealed to us until after graduation. We focus on the mathematically defined and certain world of software. It either works or it doesn't. If it doesn't work, we can step through our code until we find the logical flaw in our thinking, fix it, and the code will run. This world is even described in terms of rigorous, formal methods, where we attempt to prove that our thinking is right before we act, by rigidly defining our systems and proving their efficacy. It is this world we strive for, the world of mathematical certainty. Here we feel happy, we feel rigorous, we feel comfortable.

There are many issues with this way of thinking, not least the halting problem and the impossibility of proving that code is free of bugs—there would be no mathematical certainty even if we could ignore the business world. The part of our work that causes the most issues, however, is always outside the code. It is the business environment, the stakeholders, the constantly changing requirements, and the fact that we are building software for a future that we cannot describe or even understand. Here, there is a great deal of uncertainty. The requirements that we receive are often caged in uncertainty and ambiguity. To solve this, we use hedges like requirements engineering, risk management, scenario planning, and edge case analysis, to try and create a state, or at least a sense, of certainty. Sometimes we even use numbers and calculations, like probabilities and impacts, and we multiply them

in colorful spreadsheets, to try and give a veneer of mathematical credibility to our attempts to bring certainty to deeply uncertain environments. The overarching belief is that certainty exists, and the uncertainty is just something on the edge (hence the term *edge cases*), a minor inconvenience that we will eventually get rid of. The Gorgias of the software industry learned quickly that the confident assertion of certainty is the fastest way to get people's attention (and money). This clash between the certainty of code and the uncertainty of the environment it runs in is central to understanding the world of the software architect.

Achieving a decent level of certainty in software is (or should be) easy. Writing, testing, debugging, and troubleshooting code relies on logic and some knowledge of how the machine works. The uncertain world of the business environment is not easily described in this way, but the industry is always prepared to listen to a pitch for a new tool or framework that promises to make it so. For a very long time we tried to solve the problem of uncertainty by blaming our users. If they couldn't tell us in a logical, reasonable way how the business environment worked, so that we could easily transcribe it to code, then they had failed, not us! We called this magic method 'scoping' and put a lot of energy into protecting ourselves from their ignorance. We introduced ideas like SMART¹ requirements and analyzed stakeholder language for ambiguity in the hope of removing it. We sometimes demanded that our users tell us what their non-functional requirements were—despite the fact that they had no idea what that meant, and that even if they did, there was no way of actually knowing. The use of specifications tied to legal contracts protected us from the inability of software architecture to consistently deliver. The business model this created was enviable—when we failed to deliver something the customer got to take the blame and pay us again to build it a second time! This inability to deliver was described in 1968 at a NATO

¹Doran, G. T. (1981). There's a SMART way to write managements's goals and objectives. *Management review*, 70(11).

conference in Garmisch, Germany, as *the software crisis*². This conference tried to grapple with the problems of why software was always late, rarely worked to specification, and often cost more than predicted. A huge part of the problem, it seemed, was the idea that any of this could be predictable. A pertinent question is why we, as an industry, ever believed that it could be predictable?

²Gibbs, W. W. (1994). Software's chronic crisis. *Scientific american*, 271(3), 86-95.

Reductionism

In philosophy the doctrine that holds that everything is predictable and therefore controllable is called *determinism*. This school of thought believes that, if we understand the initial conditions and the natural laws of a system, it becomes possible to predict the future before it happens. This belief has its roots in the success of Isaac Newton's Classical Mechanics and Descartes' focus on certainty in his *Discourse on Method* and *Meditations on First Philosophy*. Newton's ideas allowed a stunning level of accurate prediction of the future movements of heavenly bodies (and earthly ones) that led many to assume that the entire universe, and everything in it, followed a similarly regular and predictable kind of natural law. A key feature of this determinism is the belief that things can be broken down into their constituent parts, and by studying the parts it is possible to determine the behavior of the whole. This is known as reductionism.

The software architect will usually have a background in a STEM (Science, Technology, Engineering, Mathematics) subject. These subjects are still today dominated by the ideas of Newton and Descartes. These rigorous disciplines seldom prepare the software engineer for the comparative social madness of business environments. When STEM graduates emerge from the cave in which they have studied their equations, eyes watering from the sudden brightness of the world, the imprecision, uncertainty, and difficulty of human social environments come as a shock. The only training these poor souls can fall back onto is STEM, and this creates a flux of simplistic, mechanical models of the business environment, as these STEM adventurers desperately try to bring the problems they encounter back into the only arena they understand.

The first echoes of this can be found in IBM's Business Systems Planning from 1967³, in which businesses, in all their human

³Kotusev, S. (1986). The history of enterprise architecture: An evidence-based review. *Journal of Enterprise Architecture—Volume, 12(1)*, 29.

complexity and detail, are reduced to simple processes and diagrams. Other concepts that emerged around the same time, such as Herbert Simon's near-decomposability (1970)⁴ encourage this kind of *reductionist* breakdown. This means reducing a system to a collection of independent parts and learning to understand the system through the study of those parts.

The vague divide between what can be known with certainty and what is uncertain causes an enormous amount of stress and defines the field of software architecture. As a programmer, you can live in a little bubble in which requirements fall out of the sky or are brought to you by a user, and you only have to worry about getting the code to work to the very clear specifications that you have been given. If the specification is not clear you fix it by complaining. As you progress in your career it might occur to you to try and move beyond complaining and fix the problem. Taking responsibility for this problem means engaging with the uncertainty of business domains. This is where real architecture work starts, with *aporia*. To make this harder, no two architects ever travel the same path. They learn by *doing*, over the course of long careers, that are shaped, accidentally, by the projects they work on and ideas that come along at certain points. The path to knowledge is not the learning of formulas and algorithms, but a journey of reflective practice⁵. This has the unfortunate effect of making it extremely difficult for architects to describe or agree upon the form or nature of their work.

Most approaches to software design involve reducing the business system to a small number of mechanistic constructs that in some way represent reality—by breaking the systems down into entities, processes, capabilities, or other elements. These systems are not obviously organized as distinct components. They have very intricate relationships and porous boundaries, but there is a sense in which

⁴Simon, H. A. (2012). The architecture of complexity. In *The Roots of Logistics* (pp. 335-361). Springer, Berlin, Heidelberg.

⁵Schön, D. A. (2017). *The reflective practitioner: How professionals think in action*. Routledge.

they can be broken down. One way to reduce the fuzziness of these systems is, as I have already mentioned, near-decomposability. In 1972 David Parnas warned of the some of the dangers of software's version of near-decomposability, *conventional decomposition*: the most common practices of breaking down systems according to real world process steps or grouping them by functional similarity led, he said, to brittle designs that could not cope well with changing environments. It is possible, and quite easy, to arrive at conceptual models that damage the ability of software to change as the environment changes. It is this risk that we must employ architecture to counter, making sure that the conceptual model does not lead to a software structure that makes it impossible to keep up with the demands of a changing business context.

The question of how to understand a business environment is a complex one. The simplest way to deal with a complex problem is assumed to be to reduce it to a smaller problem when seeing the world through the mathematical lens of Newton. When that reduction isn't obviously feasible, we engage in a trial and error approach with different kinds of reduction. Each of these kinds of reduction is usually accompanied by a framework, a certification, and a bunch of books and tools. This includes things like capabilities, processes, value streams, objects, or services. These arbitrary divisions make the system easier to deal with cognitively and also bring the promise of potential improvements in flexibility, performance, and other nice things. Reduction is a standard technique in critical problem solving in mathematics, but for some reason, it quite often fails to get the right results in software, or in most domains that are considered to be complex.

For reasons I will dig much deeper into, the tendency of the beginner architect to reduce all problems to simple, mechanical models is instinctive. This impulse is easily explained as the method by which the STEM graduate has been taught to solve all the problems they ever met. This has resulted in the software crisis. At the root of this reductionism lies a belief in determinism. But

why do STEM people all make this mistake?

Walk 3: Determinism, Reductionism, Logic and the Birth of Computing

The code that we work with in computing systems is deterministic. That is, we control it, we can predict it. The act of programming is about making the code behave in a deterministic way. There are many things that make this difficult—bit flips, bugs, etc.—and a large part of programming is making this volatile world work in a way that is deterministic enough for businesses to rely on it. Our code is deterministic because if it was not, it would be useless. Code is logical and therefore easy to reason about, easy to control, it is easy to predict what will happen through the use of logical reasoning. As programmers, the job we are given is not simply to write instructions for the computer to execute, we must also decide what instructions to write. This is where it gets difficult. In order to grasp exactly what instructions to write, we must listen to stakeholders, analyze markets, observe competitors, and think about risks in the future. This is the uncertain world of Walk 2 and it is not at all deterministic. The first instinct of the programmer is to try and describe this world in a familiar way, by making the real world resemble the world of the computer. This involves *reducing* the world around us to the kind of logical statements that can be translated to computer code. To understand where this impulse comes from and what it leads to, it is necessary for us to take a little journey through the history of computing.

The Birth of Computing

While Charles Babbage's difference engine in 1820-ish is considered the beginning of modern computing, and Ada Lovelace's subsequent work the beginning of software, most accounts start with Alan Turing's 'On Computable Numbers' in 1936. As a work in Pure Mathematics, this 36-page document is rarely read by programmers, but it lays the groundwork for coming up with the necessary foundations for efficient computations and for software. In historical accounts, Turing is often accompanied in the narrative by Von Neumann and Church, also mathematicians, who created similar theories and ideas about the physical architecture that eventually allowed for Turing machines to leap from the pages of theoretical articles into real world machines. Turing would go on to create a fundamental idea in the birth of computing, now known as the Church-Turing thesis. The Church-Turing thesis states that all problems that are computable can be solved by a Turing machine. That is, anything that can be computed by a human can be computed by a Turing machine. A Turing machine is a theoretical model very similar to a modern computer; in fact modern computers are still based on this idea. In today's world there are very many programmers unfamiliar with these early works and concepts, and these ideas are worth exploring. Turing, although not a philosopher, has had a lot of influence on modern philosophy of mind, especially his 1950 paper on "Computing Machinery and Intelligence"¹. These ideas hint at the fact the human mind itself may be computable, bringing forward the idea that humans are in some way machines. Later thinkers built upon Turing's ideas to suggest the entire physical universe is computable. It is here that the connections to determinism can be made, not necessarily in the views held by Turing himself, but certainly by many who interpret them.

¹Turing, A. M. (1980). Computing Machinery and Intelligence. *Creative Computing*, 6(1), 44-53.

Turing's work heavily leaned on the thinking of Bertrand Russell, and their ideas influenced the early development of computers. It is obvious that without this focused perspective on logic and determinism we could never have arrived at the computer. But, in this book, I argue that philosophical assumptions based on logic and determinism have concealed the problems that eventually caused computing to unravel with the software crisis. As computing emerged from Turing's machines and into the real world, computer scientists and programmers would need a way to reduce the world around them, the world of users, business functions, and language, to the logical constructs of computer instructions. Three thinkers who held great sway over discussions around logic and language at the time were Frege, Russell, and Wittgenstein. These philosopher's ideas were the only alternatives available to programmers taught only mathematics and computing, and they have greatly influenced how software architects create conceptual models.

These thinkers are mostly left out of the history of our discipline. Turing worked at Cambridge University, at the same time as Bertrand Russell and Ludwig Wittgenstein, two philosophers and mathematicians who would shape the philosophy of their time and who were central in forming the concepts that made it possible for Turing to invent computing.

Both Russell and Wittgenstein were important for a group of thinkers known collectively as the Vienna Circle who defended a philosophy called *logical positivism*, one of the early expressions of the tradition of *analytic philosophy*. Positivism was an idea first espoused by August Comte, who held that only data from experience, reason and logic were valid sources of knowledge. Comte promoted his positive philosophy as being able to bring the methods of the natural sciences to bear on political and social systems. Where the ideas of positivism are present, one will often find determinism as an underlying belief and reductionism as a method. It is this school of thought that leads the software engineer to be tempted to try and grasp the business context with the same

thinking as writing a compiler. Logical positivism took this idea to extremes, holding that our very language should only allow for statements that are verifiable through observation and that many of the problems of philosophy could be resolved entirely by paying more attention to the logical structures inherent in our language, excluding what they deemed whimsy and nonsense.

Before the Vienna Circle, similar ideas had first been espoused by Gottlob Frege in the late 19th century and, in the early 20th century, by Bertrand Russell. Russell's project, *Principia Mathematica*, was aimed at producing a language which could completely mathematically describe the world. Russell was attempting to solve a logical problem which came to be known as Russell's Paradox, according to which the set of all sets that are not members of themselves cannot form a set. He unwittingly laid the foundations for modern computing with this work.

Frege and Russell worked on a set of ideas known as *logicism*, the idea that all of mathematics could be reduced to logic. This idea was fundamental in propelling the world of computing forward. Russell's theory of types, and ideas such as *logical atomism* painted a picture of the world as a hierarchy of types, all carefully stated to be logically correct. Russell believed in the construction of a logic that could describe everything in the world, that would reduce the problems of noise in natural language, getting rid of ambiguity and fuzziness. This theory of types would later inspire Church's lambda calculus, another foundational idea in computing.

Logical atomism is akin to trying to reverse engineer the programming language that we and the world are running, producing something that ultimately would be able to run on a computer. Imagine if you could just take your stakeholder's language and turn it into code! Every generation of programmers has heard this promise, and Russell and Frege were probably the first to suggest the possibility, albeit long before the invention of the computer. Russell's project ended in philosophical failure, but most of modern software engineering is unwittingly attempting the same thing with

much less rigor and understanding and then wondering why we consistently fail. Russell's logical atomism can be seen in the construction of software entities, database tables and the way that architects are taught to analyze requirements even today. These ideas are consistently reflected in the architect's conceptual models.

This thinking causes programmers to view the world in a very narrow way. In the *Tractatus Logico-Philosophicus* (a very heavy read even for philosophers) Russell's student Wittgenstein, stated that anything which couldn't be spoken of clearly should 'be passed over in silence'. He dismissed anything that could not be formally proven as mysticism. Wittgenstein's ideas differed in some important ways from Russell's, and I include him here because he is arguably the most famous of these analytical thinkers, and because so many architects I know describe him as their favorite philosopher. Frege, Russell, and Wittgenstein all differ slightly in how they view the world, but all three are in agreement that logic and language are at the front and center of understanding it.

Whilst logical positivism fell out of fashion in philosophical circles, this was the only philosophy available to the new generations of programmers emerging from the universities and beyond, for whom the entire universe had logic at its centre, just as Russell believed. The ideas were short lived. Wittgenstein changed his mind about his strict interpretations of language as a mathematical entity, and the Vienna Circle lived to see the project deemed a failure. The 1951 paper, "The Two Dogmas of Empiricism", by W.V.O. Quine signaled the death of logical positivism. Quine attacked both the reduction of the world to statements and the reduction of those statements to logical units based on sense data. Reducing the complex world around us to a set of logically tight statements was subsequently abandoned, declared dead by 1967, except of course, in programming languages, which had to do this by necessity.

The obsolete ideas of logical positivism thus continued to rule inside the chips that were appearing in increasingly powerful

computers. Turing's ideas on computability and the computable mind and universe are mainstream even today, often with a harder deterministic view than Turing can be shown to have held. In fact, with recent narratives on artificial intelligence, they appear to be getting stronger, although these modern narratives have more in common with Gorgias than Turing's careful thought. The work of Russell, Wittgenstein, and the Vienna Circle spoke more to the machine than to the human, and this meant that the spirit of their work would infect the future of computing, even beyond the realization that something wasn't working at the 1968 Garmisch conference. Any attempt to logically analyze a problem involving information based on human thoughts, opinions, or language will include ideas or shadows of ideas from these thinkers and their contemporaries. Logical positivism may have failed to reach its lofty goals, but there has not been a replacement when it comes to mapping human ideas about the world on to logical programming languages.

Therefore, logical positivism is always there, in the background, embedded in the thinking of almost every computer science professor and student, who remain completely unaware of the philosophical implications of this worldview, and unaware that they even carry them. This accidental, inherited worldview shapes and forms our architectural tools and the architectures we design.

Requirements “engineering”

As computing power grew and became more widespread among businesses, it became necessary to specify problems in everyday language from non-technical people to technical people. When the software crisis started to become apparent, the approach was to try and manage the language of these stakeholders. Wittgenstein had taught us that most problems that couldn't be solved properly were simply a matter of linguistic confusion. While Russell's and Wittgenstein's original ideas of a reduced language amenable to logic had long since disappeared, this thinking still held sway in how the industry mapped from stakeholder wishes to computer code, often through an intermediary known as requirements or variations on formal specification that aped the ideas of Russell but never matched their logical precision (as they simply couldn't).

Perhaps inspired by Wittgenstein, the industry doubled down on controlling language through what was commonly called requirements elicitation or requirements engineering. Requirements engineering approaches sought to remove ambiguity, freeze language, make statements atomic and rigid. As the philosophical world moved on from logical positivism, the world of software remained dangerously stuck and the software crisis was, at this point, inevitable.

The result was software that worked logically but couldn't be anchored to the real world, often witnessed by the difficulties software had with changes in the real-world conditions the programs supported. As computing advanced and was expected to solve ever more intricate human problems, the challenges of constant change started to show up. Dijkstra's famous 1968 letter 'GOTO Considered Harmful' highlighted the fragility of the procedural approaches to writing code of the time, and several ideas came together, including Böhm and Jacopini's ideas on structured programming, to try and influence the structure of software programs to cope with this change. This forced the industry to start dividing

code into blocks with the idea that there would be patterns of arranging these blocks that would make the programs more amenable to keeping up with the world around them.

Russell's type theory and logical atomism played a role in how the industry would attempt to describe the world around them as input to decisions about the size and shape of these blocks. The determinist belief that language and logic and reality were somehow related—if only we could reduce language to the logical *predicates* (statements) and get rid of all the fluff and confusion we ourselves caused, tied nicely into the naïve belief that computer programs and their structures were an adequate way to express the reality around us. This is reflected in the initial methods used to decide the boundaries of these blocks. Even today developers are taught to use the appearance of nouns and verbs in stakeholder's statements to shape the structure of programs, and this is a hangover from the philosophical projects of Wittgenstein and Russell, and by extension the logical positivists. Modern takes on software engineering, such as Domain Driven Design, return again and again, just like analytic philosophy, to the problems of language and the need to control it.

Logical positivism leads to aporia, as the world simply cannot be reduced in this way. Logical positivism fails on a philosophical level because of the many internal contradictions in its own thinking, the most important revealed by Quine in the “Two Dogmas” article. Its contribution to early computing is enormous, but the extension of the ideas to requirements engineering and conceptual modeling leads to static, brittle conceptual models that are incomplete and very vulnerable to changing environments. However, if you allow some fuzziness and apply the methods of reduction, reducing stakeholder language to atomic statements, assuming that their language adequately captures the underlying business reality, you can gather enough information to begin building a software system around it. It very often won't be the right system, but the business tendency to favor action over inaction kept the ball rolling in this

direction for decades.

Coping with change

Reducing the world to a set of precise logical statements introduces fragility in the face of change. Nassim Taleb packages this problem nicely as *the Platonic Fold*, where reducing something to a model is like folding a piece of paper, making it smaller. This hides details in the “fold” that later emerge as surprises to those interfacing only with the reduced model. In the 1971 paper mentioned earlier, Parnas recommended that programs should be designed according to what is likely to change. This is a strange idea for modern software developers. There has been a growing frustration with our inability to say what is going to change in business environments since the 1990’s. We found that there was simply no way of capturing requirements up front, as things changed in between conversations. As we have seen, attempts to plan and be precise with language and the development of rigorous methods for requirements analysis, have continually failed to describe the world adequately, so the advice to design boundaries according to events we can’t see appears to have limited usefulness.

The underlying positivism and determinism in computer science explains this kind of flawed advice. If our starting point is one of determinism and logical positivism, we expect to be able to know what will change and state it clearly enough for a computer to understand. But it never works out like that in real life. Even Parnas, who recognized the problem with simplistic reductions and decompositions, was tied to a deterministic way of thinking in which change could be known in advance. This is the heart of the software crisis—that we must know the unknowable to design software.

There are many problems with the reductionist view of the world, and many tortured philosophical arguments for and against this view have been made over the years. The argument that is most important here is that *reductionism simply doesn’t work for business contexts*, because listing and understanding the elements

and their behaviors in a complex business environment, either as causes and effects or structures or requirements, is simply not possible. It may well be the case that one day it will be possible to map out all of the elements and their causal relationships in the universe and all their possible effects, but the likelihood of my discovering how to do so before I need to present an architecture at next Friday's board meeting is very small, and I should probably focus on the more pragmatic approach here.

How then, should the architect cope with this set of conditions, that contradict the architect's educational training, that contradict the default worldview of the computer scientist? The first sign that something is wrong will be flux, chaos, and disagreement. The natural response to this is to try and get rid of the chaos, to hold the world still.

Walk 4: Chaos and the Philosophies of Stillness.

In *The Birth of Tragedy*, Nietzsche complains of the dominance of the logical approach in Western philosophy, the rejection of flux in favor of rigidity, of Socrates' search for the simplicity of rigor and logic. The major problem that I have outlined so far is that software architecture tends to try to hold the world still to arrive at a design, even when the world itself is anything but still. This need for stillness can be traced back to Dijkstra and back even further to Turing, Russell and Wittgenstein, and all the way back to Plato and pre-Socratic thinkers. It turns out that this struggle was not unique to the 20th century, nor even to the Ancient Greeks, but can be seen as a defining feature of Western thought. In the approximately 2500 years since the philosophies of Ancient Greece started to develop, there have been a lot of changes in our way of thinking about the world, and the impact of these shifting ideas can still be traced in our modern thinking. By uncovering a path through these ideas, we will find that many of the swings and lurches within the history of software engineering are almost mirrored in the development of Western philosophical thought through the centuries.

Widely regarded as the first Western philosopher, Ancient Greek mathematician Thales suggested that the world was made of water. He was the first philosopher to look for an *arché*—the principle on which everything else is built. The word *arché* has some connection to *archontekton* (*master builder*), the original Greek term that later became the English word 'architect'. Thales sought a building block, and while he was obviously wrong about the root of everything being water, it was the impulse, the instinct, to imagine that there was something underlying and unifying everything we see around us, that arguably drove Western thought and lies at the

root of our modern scientific understanding of the world. This was the first step toward reductionism.

Following a similar approach, Democritus, was the first to posit that the world was made of tiny little things called atoms, an idea that underpins physics to this day. He was also one of the first determinists. The impulse in Ancient Greece to reduce the world to an *arché* is a technique that has given us great benefits, as it allows us to gather round a concept and work from there. The choice of an *arché* informs how we will build up our model of the world, what we will reduce the world to, and greatly influences how we think.

Software owes its existence to the study of logic, and assuming logic was an *arché*, as Russell did, was a natural place to start. Early programmers simply mapped straight from the business problem to procedural code, and the real world was expressed in raw programming logic. However, problems emerged rather quickly. Dijkstra's 1968 article on GOTO can be seen as a major signpost of the collapse of the reduction of the business world to logic—the inability to cope with changes. Logic was simply not enough. This forced the software world to start thinking seriously about *arché* beyond logic for the first time. There had already been considerable work done on this, but it was Dijkstra's letter that made the industry sit up and take notice. This can be seen as the beginning of architecture, a mode of thought beyond programming that was going to be more focused on the relationships of the code to the world it executed in than the optimal execution of code based on hardware limitations.

Software has searched for its *arché* ever since. We have had many different ideas—first the machine was the *arché*, then we separated the machine from the software, and we got the procedure, the module, Jacopini's and Böhm's structured programming and blocks, the actor, the object, the service, the microservice. How we chose to divide the software into blocks was informed by the environment, by the computer systems being built, and by the worldview of those tasked with finding better ways to build computer programs.

Very often the arché was driven by the technology, which, in turn, has shaped how we see the world. Separating blocks of code from each other to make them easier to change, inevitably leads to an architectural view that sees the world as being made up of interacting objects. This constitutes a very strong (but accidental and unconscious) philosophical position, which our industry has taken on at a whim.

As the industry moved toward structured programming it embraced a stream of different arché over time. Early attempts had their roots in mathematics, such as Hewitt's actors. Eventually new attempts were made to define the building blocks of software that had little connection to mathematics and came from industry practice, trial and error, and real-world needs. The most effective of these, the service, has little mathematical backing and is grounded in the provision of clear boundaries which is fairly vague and undefined, it is based more on practice and reflection.

Each time we break down the software into a new type of unit, we essentially adopt a different worldview, a different reduction. This process of shifting worldviews has been ongoing since the 1960s. Each new swing of the pendulum, each new way of breaking down our software components has been driven by real problems in producing software. For example Dijkstra's experience of the exposure to change caused by the GOTO method, or the introduction of networks that led to distributed computing, or the shift to cloud computing infrastructures. The history of our discipline is often told emphasizing these shifts as major inflections in the structures of our solutions. But what this story omits is the reasons why we choose to break down software in a certain way rather than another. By telling a different story, one that follows the links between the evolution of philosophy and software, we will see that a lot of crucial decisions in our discipline have been guided (or misguided) by a need to bring stillness to the business context and a lot of the problems this entailed arose because these attempts to keep things still have failed miserably.

A major issue that arises here is the fact that many software architects fail to realize that their thinking, their choice of unit, of an *arché*, is not the same as their colleagues, and it is not the same because, philosophically, they view the world differently. This means that discussions are often heated, rhetorical, and ultimately aporetic.

The problem at the heart of software architecture is that the software structure is more static than the environment it will execute in. The business context moves faster than we can reliably change the structure of our code. The conceptual structure that we build must be static to be implementable. We would obviously have chosen to make software flexible and instantly changeable if we could, but this was much too difficult. Instead, we chose the much easier path of freezing the business context in a conceptual structure and protecting this with business contracts and up-front specifications. This has allowed us to work with a sense of financial stability, but it has failed to solve the problem. The business context still shifts over time, and the architecture will need to be constantly patched and updated as this happens. Whilst software, deterministic and mechanical, is easy to carve up at its joints, the business context will not prove so easy to reduce. Time and again architecture makes the mistake of bringing the *arché* of the code structure to the business context, dooming the architecture to failure as the business context moves and shifts. This leads to progressively weaker and flakier architecture over time. Where did we get this idea from? It turns out that it is a very old idea indeed.

Parmenides, Plato and our obsession with stillness

The pre-Socratic thinker Parmenides is very interesting for the purposes of our story. Parmenides believed that the world was “one”, meaning that it was just one thing, and that it was still. He believed that our human perception of change was a delusion and that the universe was in fact eternally and perfectly still. Plato, some years later, while critical of some aspects of Parmenides’ thought, would seize this ideal of unchanging stillness as a foundation to his own thought. Plato presented a philosophy that aimed to unveil the world of eternal and unchanging *Ideas* or *Forms*, beyond the sensory world of change. He claimed that change was an illusion of our unreliable senses, and that there was a path to knowledge that would quiet this noise and bring us to the point of stillness, becoming wise enough to understand these unchanging Ideas. This stillness is of course something that the software engineer would very much like to bring to bear on the business environment in which software must run. If only we could reach something like this Platonic wisdom, then maybe we could actually get around to writing some code!

Over two millennia later, in the 20th century, the Austrian-British Philosopher Karl Popper argued that because Plato’s life was full of strife, war, drama, and constant change, he may have found solace in the idea of eternal, unchanging Ideas. We could argue that it is perhaps this same urge that drives architects to yearn for calm and stillness amid the chaos of IT projects.

Plato saw change as decay. For Plato, sensible things (things which we experience through the senses) are imperfect copies that drift from their perfect, still, unchanging, form. Change according to Plato is dangerous, and stability is divine. A similar idea appears in change management frameworks, or preservation of state in IT operations, or the enterprise architect’s predictable, frozen “To-

Be” architectural future state. The agile movement’s “embrace change” slogan was therefore revolutionary in a way that probably wasn’t intentional. Developers, who had long suffered in the trenches, realized that suppressing change in the name of imaginary but convenient stillness was not in their best interest, and led to constant revisions and false starts. But many stakeholders remain more comfortable with the idea of ideal or perfect forms and the suppression of change. Some development methodologies, including Parnas’ methods of encapsulation, encourage the suppression of change and use of concepts like “blast radius” to contain change.

In one of his most famous dialogues, the *Republic*, Plato allowed his view of the world to shape how a city state should be designed and built. Similarly, enterprise architecture builds guidelines, reference architectures, and review boards, and this is Platonic in its impulse. The early 2000s vision of the Enterprise Architect, as some kind of business and technology guru, mimics Plato’s concept of the philosopher king he develops in the *Republic*, which, just like the vision of enterprise architecture, never truly materialized.

For many who have studied philosophy, Plato is their first introduction to the subject and his place in Western thought is almost sacrosanct. Criticizing Plato in any way must be done cautiously. There are many ways to read and interpret Plato. Karl Popper critiqued what he saw as Plato’s totalitarianism in the *Republic*, owing to Plato’s ban on poetry and more controlling tendencies. Other interpretations state that this *obviously* wasn’t what Plato meant but that *Republic* is a psychological allegory rather than a political work with real recommendations. I have no wish to indulge in a century old war where interpretations are juxtaposed, but find some of Popper’s ideas interesting given my own suspicions of the relationship between the stillness of Plato’s forms and the rigid control espoused in the ideal state described in the *Republic*.

How does this thinking turn up in software? Many software architecture methods work by holding one thing still, focusing on

only it, and allowing the other parts to move or ignoring them. A fantastic example is the 1974 paper “On Scientific Thought” in which Dijkstra defines the famous concept of separation of concerns. This paper dominates software architecture thinking (although few have read the original!) and it works by holding one thing still at a time and viewing the world from that perspective. This is a Parmenidian approach! Another approach consists in holding everything still and allowing only one aspect to move. I often recommend George Polya’s book *How to Solve It* to beginner architects, as a tool to sharpen their critical thinking skills. This book, which helped me greatly in my early career transitioning from Pure Mathematics to computing, embraces the second technique: holding everything still and allowing only one thing to move. But in 1971, Parnas took this idea to its extreme and suggested that the thing to be held still is change itself—that change itself can be constrained, encapsulated, frozen in our code. Waterfall approaches seek to suppress change through planning. Parnas tries to suppress change by predicting and encapsulating it. Risk management, change management processes, and much of the agile movement are struggling under the weight of their own Platonic impulses to hold change still by isolating it or embracing it. Plato’s view of change as decay, as evil, and stillness as divine is still very much mirrored in our approaches to software engineering.

Software engineering was therefore shaped by its urge to contain and control change. But it was also shaped, perhaps even more so, by the trauma of its repeated failure when the conceptual structures of software architecture were confronted with the real world. The software crisis that I mentioned in Walk 1 can be boiled down to the failure of this expectation of stillness.

The need for control in enterprise architecture, the attempt to turn everything into a simple set of processes that would universally work across all business and industries, was an attempt to bring stillness. The result, as we now know, was chaos. The agile manifesto was a reaction to this need for control. It rejected the

world of up-front planning and large, detailed specifications in favor of iterative approaches to software development. Over two decades after the manifesto was released, it is not uncommon to see claims that the agile movement has died. In many cases, the agile movement has become the victim of attempts to keep *agile* still and reduce the uncertainty in the process itself, as ideas and frameworks appear to make agile repeatable, controllable, and still. The pattern is repeated throughout our short history.

No matter how much time we spend defining an abstract model that describes reality, and no matter the beauty of the arché or our devotion to it, we will experience something called model drift—in which reality starts to drift away from the structures of the model, rendering the model incomplete. Software structure, architecture, is the implementation of this conceptual model in code. Since software structures are static and must also contain a model of the world they engage with, then the search for a static model that can both be coded and at the same time represent the world is inevitable. This automatically leads the architect astray, seeking the same kind of stillness in business reality that Parmenides and Plato failed to find. Architects will seek this stillness in different directions—in the Forms of Plato, in managing language as the logical positivists, in removing intuition as Frege, or seeking certainty in a still foundation as Descartes. All of these philosophical projects would fail—they are undoubtedly, as Whitehead noted, footnotes to Plato, who 2500 years ago in the dialogue *Parmenides*, toward the end of his life, made the first arguments for why the ideal stillness he had described was not possible.

The Accidental Philosophy of Software Architecture

My journey through the history of philosophy has led me to the conclusion that the constant debates between different camps in the IT industry about the usefulness of certain tools and which method was better than the other could never be resolved because there was a fundamental difference in the philosophical thinking between many of them, but no one had the resources to understand that these differences were irreconcilable and incommensurable. Determinists railed against anarchists in terms of methods, using anecdotal evidence and rhetoric to drive what were essentially social campaigns for acceptance rather than scientific explorations of what worked and what didn't work. While attempts to put emphasis on empirical aspects of software research have been prevalent in recent years, they struggle to be relevant in the real world of software engineering. Empirical studies and attempts at generalization of wildly different projects driven by different teams in different contexts were simply impossible—once again, a complex problem, which is addressed by naïve empiricism, rhetoric, and anecdote.

Once I started to look at architecture as a philosophical process I started to see patterns in the thought of software architects and developers that had obvious parallels to ideas and methods in the history of philosophy. I called this the *default philosophy of software architecture*, because most architects carry these ideas in some form but are unaware that they do, and have even less awareness as to why. This default philosophy is very clearly a philosophy of stillness.

The following aspects can be found in the underlying philosophy of most approaches to architecture:

1. Essentialism

2. Causality, mechanism & determinism
3. Structuralism
4. Cybernetics
5. Trade-offs
6. Modeling

All of these things are examples of a continued reductionism. Over the next few chapters I will outline how architects have embraced a number of philosophical ideas that were never consciously taken onboard but were the result of the search for stillness in the world of business flux.

Walk 5: Essentialism.

Architecture involves the design of software systems. This design means defining and linking software components that will have some kind of relevance to the real world. In the software, we create an executing model that allows us to update data stores, make calculations, or change the record of what is happening in the world. These software systems are just mathematics running on a chip, but they will also, in some way, represent the world they execute in. When we want to record the details of a particular transaction, we will need to create structures in our code that in some way relate to the elements of the real world. It is impossible to build a software system that represents customers, products and orders without the software system and source code having some reference to those customers, products, and orders. The software represents the real world, even though the job of the computer is not to mirror the real world, but simply to keep a record of it, to perform some calculations, and perhaps inform some other system about what is happening. The software is not the real world, but it can be easy to mix the two up as we seek to bridge them.

The major difference is of course that the software cannot represent every aspect of the real world, and it cannot represent things which the architect or developers of the software do not yet know. The software is actually a very weak representation of reality. It is a separate, new thing that is not the same as the world it mimics. It is, however, amenable to the ideas of the logical positivists, since it's just binary math.

When we create structures for our computer code we are doing two things. The first is that we are creating a set of instructions for the processor. These instructions are inescapable. We must adhere to them. These instructions will create structures in

computer memory, some of which will represent concepts in the real world business system. As the real world shifts and changes, the instructions must also change, hopefully without requiring the entire application to be rebuilt and without damaging the system. This is where architecture comes into play. Advances in software engineering in the 1960's and 1970's, Kay's objects, Parnas' information hiding and Dijkstra's structured programming have helped abstract away the details of computer memory and allow programmers to think about higher level abstractions in source code, that often seem to map onto the real world around us. How we choose to describe the world around us is the conceptual structure described in the first chapter, which is then used to create the software structure in source code, which compiles to create the instructions that ultimately create structures in memory when the application runs.

Platonic Forms and Abstraction

Plato distinguished between forms (abstract ideals) and sensible things (what we perceive with our senses). He posited that the forms existed in a kind of heavenly realm, as the abstract ideals of earthly things. Earthly, sensible things are defined by their heavenly forms, they are less perfect copies of them, or simulacra. Anything black takes part in the form “blackness”, anything beautiful takes part in the form of “beauty”. In the dialogue *Parmenides*, Socrates discusses the existence of forms but also voices some reservations about this idea, showing that if it were true, there would to be forms for lesser things like mud, hair, and sticks. This is the major flaw of the theory of forms, an endlessly growing list of perfect forms that eventually becomes as large as the number of things on earth.

In trying to define these forms Plato was seeking to define the essences of things. An essence is the core of something, what makes it “it”. The belief in essences is called *essentialism*. Appearance is how we perceive the thing. No matter how much effort we put into defining essences something will always come into view that calls that essence into question and finds our definition of it lacking. Trying to close this phenomenal gap has been the effort of philosophers for millennia. Parmenides said we should ignore the differences as noise and focus on the essence, in the belief that the whole universe was simply one thing we could reduce everything to. As such, he was the first Enterprise Architect, choosing to elevate the model and sweep reality under the rug. Plato followed Parmenides’ lead and came up with a scheme to preserve the idea of essences—his theory of forms. The supremacy of essence over appearance permeated the history of Western philosophy. Aristotle, a student of Plato, disagreed with his mentor but carried on with the development of essentialism with a slightly different view. He rejected the heavenly forms but still sought to find and define the essence of everything around him in the form of

genus, or species categorization. He described *universals* instead of forms. These universals were a part of the things around us, not prior to them. A famous painting by Raphael “The School of Athens” shows Plato pointing to the sky and Aristotle to the ground, reflecting their beliefs about where the essences existed. Aristotle’s ideas held sway over Western thinking for centuries. He introduced formal logic, categories, and is held in high regard even today. His ability to work across multiple disciplines, and even invent most of them, is astonishing. Aristotle’s work in biology defined his essentialism. It is employed through the categorization of genus, and continued until Darwin’s *Origin of the Species* took an approach that was a stark diversion from essentialism in the 19th century.

Abstraction, as described by Popper, is the movement from particulars (actual things i.e., Felix) to universals (generalizations of actual things, i.e., cats). When we create an abstraction, we also subscribe to the theory that a certain group of things have something in common. The idea that calico, Siamese, Birman, and tabby cats are all cats might seem like common sense but this ultimately boils down to a theoretical decision. Things become slightly more questionable when we include tigers and lions into the cat category, because they are quite different from wee Felix, but the theory is widely accepted since they share some kind of *essence*. These theories are not always fixed. Aristotle, the first person to systematically categorize animals into species, categorized bats as birds, and this theory turned out to be a little wrong.

Bertrand Russell spent a great deal of time on the problem of universals in his book “*The Problems of Philosophy*”. Russell’s work is interesting when thinking about our conceptual models of the world because of these ideas on *universals*¹. He was quite critical of Aristotle’s vague descriptions of what universals were. *Universals* are generalizations over particular things*; for example the universal “whiteness” refers to a property shared by all things

¹Russell, B. (2001). *The problems of philosophy*. OUP Oxford.

that are white in the world. How these universals are handled is critical to the software architect's attempts to construct conceptual models of the business context. Is the whiteness of the snowflake the same as the whiteness of the polar bear? Does the whiteness exist before the snowflake forms, or after? Following this, is the security of a photo sharing application the same as the security of an application running in a nuclear power station? Is security something that exists before an application is designed? Does "security" exist somewhere, somehow, outside of our minds? Or is security an abstract, blanket concept that represents a number of properties that are different in every single project? You can see immediately why the architect's perception of universals will affect the conceptual models produced. Russell's logical atomism implies that these universals exist, or subsist, in the world, that they are necessary for our logic and language to function, and that our language can be reduced to a formal logic that makes clear the relationship between all things in the world.

How this turns up in software.

Plato's theory of Forms is not something that a lot of philosophers genuinely believe in, and even toward the end of his life, as in *Parmenides*, Plato himself was having doubts. However, the most faithful implementation of these ideas can be found in object-oriented programming. Here, Plato's ideal forms are found in class definitions, with each instantiated, running object, taking on many of these ideal forms through polymorphism and inheritance.

We could argue that software is the closest we can get to Plato's ideal forms. These are abstractions that exist in another place, in this case computer memory, and define the things in the world. As architects we have seen the utter chaos that occurs when the problems detailed by Plato himself in *Parmenides* turn up in our code. An explosion of types, the constant if/else additions to universal properties as they vary in the real world as soon as we define them in code. The software architecture profession may well be that which proves Plato's worries about his theory of forms in *Parmenides* to be well founded.

The mechanism behind Plato's Forms and Aristotle's universals are often referred to in software engineering as abstraction, and this is an important human skill and something most developers learn by simply doing. The ability to abstract and generalize the overwhelming diversity of reality makes our lives more manageable and easier. In the real world, abstraction happens quickly and easily. For example, we have an abstract idea of what a cat is so we are not forced to ask ourselves what each individual cat we encounter is. In software, we tend to use abstraction to reduce the amount of things we need to represent, to reduce the effort required to reason about certain things. Abstraction is a way to reduce the world around us, to make it manageable by ordering it, and occasionally, to make it still. Or as Dijkstra puts it:

"The purpose of abstraction is not to be vague, but to create the

conditions in which we can be absolutely precise”²

Both Dijkstra and Plato have the same goal; to use abstraction to reach a higher level of wisdom. Abstraction is a way to reduce the system, make conversations more effective, and make everything more precise. The use of abstraction takes care of the complexity of the everyday world, making it still, making it nothing more than a series of simple predicates (statements), which can easily be mapped to software structures. If Plato had been right about the forms, if abstraction was this easy, if (early) Wittgenstein had been right about language, if Russell had been right about logic and its relation to universals, then software would have been the easiest thing in the world to design, because the abstractions, the language, the predicates, would all have already been there, and code structures could have been worked out with a pencil and paper. Unfortunately, as they discovered in Garmisch in 1968, this was not the case. The software crisis took the form of the repeated crashing of these efficient abstractions and language against the rocks of reality. Software design was the ultimate proof that the project running from Plato’s forms, Aristotle’s categories, Russell’s logic, all the way to Turing and Dijkstra, had some major holes in it. The software crisis was the result of the world simply refusing to be static in order to be abstracted into software structures.

Most people who work as software architects share the same experience when trying to develop their conceptual models: the constant movement back-and-forth from particulars to universals. In fact, this movement is so frequent that it could be said to be in constant motion. It’s the reason we can’t trust requirements and needed a manifesto³ to try and kill them.

When architects try to create conceptual structures that describe a business environment that will then be mapped to code, there is very often a focus on the ‘things’ that appear in the business

²Dijkstra, E. W. (1972). The humble programmer. *Communications of the ACM*, 15(10), 859-866.

³Beck, K., Beedle, M., Van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... & Thomas, D. (2001, February). *The agile manifesto*

context. How we try to define and think about these things always has undercurrents of Platonic and Aristotelian essentialism. Trying to reduce the number of things leads to abstractions that hide the differences between them, seeking to define perfect essences that allows for fewer components and simpler conceptual models. When we go too far and assume that different things have the same form, we introduce coupling to our conceptual model and our architecture, and this has the effect of causing rippling and cascading changes through the architecture. If we don't try to seek any form of essence, there is a risk that there will be much repetitive code and changes must happen in many different places, which is hard to keep track of.

Aristotelian essentialism and the concept of universals affects the architecture of systems, with concepts like “quality attributes” also called “non-functional requirements” used to describe things like performance and security, as if they were the same thing in every single project and not an entangled mesh completely dependent on the current context. Together these created a way of thinking that drove the implementation of architecture as a set of repeatable objects, patterns, measurements and trade-offs. Thinking that things like customers, orders, security, logging, were all the same thing, or reducible to the same thing, across different contexts, leads to brittle architectures extremely vulnerable to model drift.

The most dangerous idea that we can adopt from Plato as architects is that a perfect, ideal form exists from which we can escape all change. From Aristotle, the idea that attributes like security, performance, maintainability, universals, are the same thing across different business contexts and can be handled in exactly the same way. These are two common traps that architects fall into. Falling in love with our essences and trying to protect them from change by expanding them until the architecture collapses, or assuming that qualities can be isolated and are the same thing across many different contexts. Both Forms and universals try to reduce the world around us by looking for things that are similar. This is very

appealing to the software architect, who wants to make the number of components as small as possible, to make the business context simpler and easier to express in software structures.

Walk 6: Causality, Mechanism, and Determinism

At the root of deterministic thinking is the idea of causality. Aristotle stated that everything must have a cause, except for the original cause, or the *unmoved mover*, God. Whenever something happens, it has a cause, and that cause in turn has a cause, and so on. When we understand the causes and the mechanisms behind them, then we should, in theory, be able to predict the future of any system. Thinking about causality is something quite intuitive. As children we learn to observe that a certain action often gives a certain result and, from there, extrapolate quite easily to a world of causes and effects. Identifying and isolating causes is one way of gaining control over a system or a thing in the natural world.

Aristotle's Four Causes

Aristotle listed four causes that explain why the world is the way it is, and these ideas are still present in modern day thinking.

1. Efficient cause.

If A then B

2. Material Cause

That the substance a thing is made of is its cause

3. Formal Cause

The shape or form of something is its cause

4. Final Cause

The purpose of a thing is its cause. This is referred to as *teleological* cause.

How does this apply to architecture?

The most common understanding of causality among architects is that of mechanistic, or efficient, causality -and why wouldn't it be? Mechanistic causality is the belief that A causes B and will continue to do so. It is nice and neat and easy to understand and you can see it in action in every compiler. Carrying this understanding of causality over to social systems that software must run in will start to cause problems, because in these systems, which we do not understand well, there are many causal relationships that are not simple. Standard approaches where architects use this kind of thinking can be seen in risk management, a kind of mathematical theatre in which probabilities and impacts are assigned to unknowable future events.

Other forms of causality that are common in architecture are described well by Ralph Stacey as formal causality - the belief in structure as the cause of the future (structural determinism), and rational causality, the belief that the future is caused by executive strategy. These ideas are also commonly found in the architect's approach to causality.

Essentialism, Determinism, and Causality

The influence of Plato and Aristotle was felt keenly right through the Middle Ages as philosophers grappled with the implications of these problems. In the modern world essentialism is different than how Plato and Aristotle used it, but their foundation was incredibly important, and their thinking can still be traced to much of how we interact with the world today. Aristotle is considered the father of empiricist approaches to knowledge. These approaches would remain relatively static until the work of Francis Bacon gave the concept new life and started the progression toward what we know as modern scientific thinking. Instead of simply trying to describe the world around us, we suddenly started to care about the how, rather than the what. Essential forms are not just to be categorized, but we also want to investigate their properties and their effects on each other and the environment around them.

In the 17th century a thinker called Descartes would have a great impact on how we think about these things. Descartes is extremely important to modern scientific thought. Descartes was an essentialist, and his work added a new twist on top of essentialist thinking. His separation of the mind from the objective world—known as Cartesian dualism—led us to believe in a world external to us which worked like clockwork and could be mapped and understood, and a spiritual world separate from that which was mysterious and divine. His ideas introduced a determinism around the natural world that was bolstered by the success of Isaac Newton's mechanics that seemed to offer a mechanistic, mathematical explanation of everything around us. The belief in a clockwork universe was awakened and this would lead us to many great successes, including the scientific and industrial revolutions, the enlightenment, and eventually, to the computer. Cartesian dualism separated the mind and the body in a way that had not been done

before, allowing us to look away from many of the great mysteries of our minds and focus only on what we could control and predict in the world around us. It would lead to the idea that the way to solve problems external to us is to reduce everything to its constituent parts and analyze the behavior of those parts.

This idea still dominates modern thinking. The separation of the world into an objective, clockwork realm and a spiritual realm gave science the freedom it needed to move around unrestricted by religious ideas but also gave scientists a convenient rug under which to sweep any renewed struggle between essence and appearance. The new idea was that change could be understood through a new kind of essence that explained it—the natural law determining interaction between objects. In this light, change itself could be seen to have an essence, and turning it into an equation was a way of making the world still through predictability. Something that appeared dynamic and alive to our perception could be made still, predictable, and ordinary by some unmoving ink on a page describing an equation.

Descartes' work—his most famous being *Discourse on Method* and *Meditations on First Philosophy*—seeks a kind of knowledge that is totally certain, that cannot be refuted. This is reminiscent of Plato's justified true belief as described in the *Theaetetus*. Descartes, like Plato, is a rationalist who believes that the way to knowledge is through reason and ideas. Descartes, however, believes in a method, a routine way of arriving at truth—and this method is focused on doubting that which immediately appears obvious to us.

Descartes is a foundationalist, which means that he believes in an arché, and in a structure that is built on that arché. In a letter comparing his approach to that of building architecture, he describes his method as seeking a firm foundation, taking away the soft sand so that something sturdy can be built on top of it. Foundationalism is something one sees most obviously in the axiomatic structure of mathematics, and Descartes was, of course, a

mathematician. The method he describes as doubting is intended to reveal to us the foundational principles, or the *arché*. This method is intended to free the thinker from preconceived opinions we all carry with us, and this might be a wise approach for many of us in software architecture, unaware as we are of the philosophical baggage we carry. This foundationalism will turn up in computing as we seek different *arché* upon which we can be certain, upon which we can build the concepts of our systems.

Other rationalists like Baruch Spinoza and Leibniz, often referred to together with Descartes as early modern philosophers, also rejected Aristotle's *teleology*, or final cause—instead focusing on mechanical causation*. Leibniz defines the *principle of sufficient reason*, that everything must have a cause. Following the success of Isaac Newton's classical mechanics, the era of modern scientific thinking was upon us. It is this kind of thinking that would eventually lead to the computer, and as such every architect who designs computer systems is touched by it.

This philosophy, which is often referred to as early modern philosophy, was a rejection of many of Aristotle's ideas, but Descartes saw his work as a development of them. (At this point of the book, I have hopped over a few centuries, completely ignoring a bunch of philosophers known as the Scholastics. These were priests in the hire of the church, and much of their philosophical worldview was Aristotelian in nature. Apologies to those who have read this far hoping to find out what impact the scholastics have had on software, there simply isn't room. They would have written their code in Assembler or C on Linux though, given their piety, love of logic, and closeness to God.)

In the 18th century, the Scottish philosopher David Hume caused a great deal of controversy when his skepticism led to the idea that there was no 'causality' that we could actually see. All we can observe, he noted, was that two things seemed to happen very near in time to each other, that there was a '*constant conjunction*' of two events that we labeled cause and effect, without ever seeing

or understanding the actual mechanisms of the causal link. To this disturbing realization, he added the idea that our process of induction, the way in which we extrapolate from a constant conjunction and assume that something will always happen, was also riddled with logical problems. Hume, however, in denying the standard view of causality, and in denying scientific certainty, still arrived at a deterministic view of the world.

If one subscribes to this deterministic view of the world, and the subsequent ability to speak with certainty about it, a simple mapping of causes in the business domain would suffice to build software. This would involve mapping, capturing, speaking to stakeholders, and figuring out what all the causes in the business domain were. For a mechanical system, this might seem easy. But it turns out that it is not at all possible for complex business domains. These maps and documents are used to define software structures. As soon as the business environment starts to shift and change, which it inevitably does, the brittle software structures crack and the architecture fails under a torrent of patches and quick fixes.

What Does This Mean for Software Architecture?

Descartes' decision to separate the mind from the material world would have major influence on philosophy, and this would turn up in how software architecture approaches the world hundreds of years later. From Descartes we get the idea that the world external to us is deterministic, mappable and understandable with certainty, that it can be reduced to a set of parts that affect each other through natural laws, which is pervasive in architectural modeling techniques. Descartes also opens the door to mystery, that the mind cannot be known in quite the same way, that there is something beyond the reach of science which we should not spend time on. This Cartesian dualism is at the root of why we separate the science of the computer from the social system that the software must operate in and focus on what is certain. In *The Mythical Man Month* by Fred Brooks, one of the most important books in software engineering literature, Brooks defines the architect as the mind and the programmers as the hands, a clearly Cartesian division of labor! Descartes' search for certainty is another attempt at stillness, at finding that which cannot move.

Descartes, Newton, Leibniz and Spinoza together brought to bear a set of ideas that would influence how the software architect would create their conceptual models in the present day. The search for causes, explained by natural laws, grounded in mathematics, with certainty built in, is added to the essences in the architect's conceptual structure. Frege will, a few centuries later, package concepts as functions and variables applicable to language and thought, and the step to programming from there is pretty obvious.

Descartes updates the essentialist worldview by focusing on determining mechanism with certainty through foundationalism. Whilst Plato's theory of forms assuredly informs the modeling of objects in object oriented programming, the mapping of relation-

ships between objects and the effects that these objects have on each other is Cartesian in nature, as it is based on a natural world of material things acting on each other, determined by natural laws. Given the software crisis, Descartes' simplistic reductionism is clearly not the right tool for software architecture.

Determinism, Essentialism, and the Age of the Machine: Cybernetics

Cybernetics is the field of study founded by Norbert Wiener in the 1940s. Originally based around the task of automating anti-aircraft guns. In Wiener's definition of cybernetics, he describes feedback loops as a concept allowing us to understand the complexity of a human behavior in which actions are continually updated in response to feedback. The idea is pervasive, including in modern complexity approaches in management that designs experiments to provide the feedback. This analogy led to the conclusion that people and organizations were simply machines, allowing positivism and determinism to flourish once again. Wiener's book would be called "Cybernetics: Control in the Animal and the Machine". This idea reached its nadir in Stafford Beer's Viable Systems Model which describes an organization as a structure of interacting pieces with feedback loops. Cybernetics has since fallen by the wayside as a scientific approach, but the prevailing attitude that people and organizations and machines share the universal of feedback loops means that it is still commonplace to describe our social systems with diagrams—humans and social systems as machines was an easy analogy to sell to the inheritors of Turing. This is an example of how we allow our current focus, our current arché, to influence how we see the world conceptually. In the 19th century Mary Shelley's *Frankenstein* reflected a worldview through the lens of a Victorian concept of science, modern conceptions of consciousness are using artificial intelligence to a shape their view of the world and the mind. In the case of Cybernetics, the world is viewed through the lens of control systems.

In 1950, the biologist Ludwig von Bertalanffy, wrote *General Systems Theory*¹, a book in which he posited that there could be a science of systems, in which the arché *system* would be understood

¹Von Bertalanffy, L. (1972). The history and status of general systems theory. *Academy of management journal*, 15(4), 407-426.

as a Platonic form, wherein we could transpose knowledge from one system to another, eventually bringing a new science that would solve the problems of uncertainty in large systems.

These ideas continued to evolve during the 1960s, with things like the systems dynamics of Jay Forrester and Donna Meadows, which reduced these complex systems to simple flows and stocks. The ideas of the biologist Maturana, structural determinism, also turn up fairly often in these discussions.

Other related thinkers such as Checkland, Churchman, Ackoff, talked about the emergence of properties, their intractability, ‘messes’, and things that could not be adequately understood. While these explorations were ground breaking for their time, they have proved limited in their usefulness in the real world and today are the mainstay of management studies often seeking to bring the machine metaphor into the organization.

Looking across these ideas, we can see that the instincts embedded in Platonic and Aristotelian essentialism kick in pretty fast, as well as the mechanism of Descartes. We immediately seek an abstraction and reduction: something must be the same if it generates the same feelings. We seek the essence in the phenomenon. Currently, the Platonic urge is to believe that there is some kind of ideal form or essence within systems called *complexity* and some people immediately set about trying to understand what that is. The Aristotelians immediately start trying to define exactly what it is that makes something complex and rush to categorization of systems as the default. The ideal form starts to take shape through new reductions—“*complex systems consist of simple agents following simple rules*”—and other clichés gather pace. The *law of unintended consequences* puts a fancy sounding name on the inevitable conclusion of intervening in a system we don’t understand. Bertalanffy’s work on General Systems Theory makes the Platonic error of assuming complexity is some ideal form of which all poorly understood phenomena partake. Weiner and Ashby assume the feedback loop is a universal which unites the animal and the

machine, but appear unperturbed by the fact that practically it only extends to thermostats and serves as little more than a metaphor. When essentialism is exposed to something beyond its grasp the result is a nascent *structuralism*—since there is no ideal form to be found in the elements themselves, it is sought in the relations, in the structure and the effects they have on each other.

The result is a body of work, often referred to as *systems thinking*. This work carries many of the same essentialist, mechanistic, and determinist elements as the practice of architecture, and the evolution of the two schools of thought would suggest a great deal of influence on early architecture by this type of thinking. Today, these ideas are mostly present in management thinking.

This mechanistic approach to the world is expressed everywhere in enterprise architecture. Processes are a form of structure that describe relations between actors and their actions. Capabilities describe relationships between the processes and the tools. Benefits Dependency Networks describe the relationships between assumptions, actions, and expectations. Yet despite these many defined mechanisms software still falls apart when exposed to reality. Everything in enterprise architecture is dependent on these twin evils of essentialism and structuralism. These ideas give obvious results when applied to simple things but repeatedly fail when applied to bigger systems. I will argue that architecture needs to free itself from this kind of thinking.

The British management researcher Ralph Stacey throws these problems into a sharp light. In his book *Complexity and Organizational Reality*,² he explores the dominant discourse of management in which new ideas are simply absorbed by the old paradigm of command and control. He talks about the dominant discourse of the MBA program, the master generalist with no specialist knowledge—this reawakens the specter of Gorgias the sophist mentioned in the first Walk. Despite the fact that systems thinking

²Stacey, R. D. (2009). *Complexity and organizational reality: Uncertainty and the need to rethink management after the collapse of investment capitalism*. Routledge.

espouses flux, and messes, and wicked problems, in practice it frequently snaps back to mechanistic, reductionist, determinist methods³. This discourse appears again in the enterprise architect, the scrum master, the change management professional, the digitalization strategist. Stacey notes how systems thinking leads straight back to the essentialism and structuralism that is prevalent in architectural thinking. He decries the existence of second order abstractions, ideal forms of ideal forms, that are produced by these approaches, disconnected from the reality they try to address.

Systems thinking has of course developed since these early incarnations, and the work of systems thinkers like Mike Jackson⁴ nowadays promotes a pluralistic, more nuanced approach, aware of the limitations of the early ideas. However, in practice, the aspects of architecture influenced by these ideas still display tendencies toward determinism, mechanism, and essentialism, whilst at the same time speaking of uncertainty and unknowability. These pluralistic approaches can often give useful outputs, in exactly the same way that traditional architectural tools do, something called the *Deleuzian Walk* which I will describe in Walk 14.

³Of course, many who practice systems thinking will be incensed upon reading this, claiming that what I am describing is not systems thinking at all.

⁴Jackson, M. C. (2024). *Critical Systems Thinking: A Practitioner's Guide*. John Wiley & Sons.

Walk 7: Logical Structuralism

Large systems, such as a business, containing many different kinds of elements, are incredibly difficult to think about in essentialist terms. But even though an ideal business does not exist management consultants will often try to sell you a vision of one. How does essentialism apply to such things? I have already shown that Descartes brings a philosophy of foundation and mechanism to the material world around us. Following this kind of thinking, the essences of the objects around us can be enhanced with natural laws that describe the relations between them and foundations that are certain. This way of thinking naturally leads to taking large business systems and tries to break them down into their smaller, constituent parts, and reason about the kinds of natural laws that might drive the interactions and effects between these parts. This immediately leads architects to focus not only on the essences that make up the system, but also on the relationships between those essences. This produces a way of thinking that leads to the description of structures, which can be seen in almost all architectural tools, and in particular in sketches and diagrams of these structures. Very often, the parts are identified by analyzing the language of stakeholders, for example using noun/verb breakdowns as mentioned earlier.

These structures are almost accidental—emerging as a result of combining a belief in essences with a belief in natural laws and the principle of sufficient reason. There is of course nothing in the work of Descartes or Newton that states that these ideas should be applicable to social structures, since their concern is with the material world. Later, Comte's positivism will make it feasible to extend this kind of thinking to social structures. Combining the

determinism, reductionism, and essentialism that I have discussed so far, the architect can now produce a structure of elements and their effects on and relations to each other.

However, it became clear very quickly that social structures like businesses were not easy to describe in terms of natural laws or simple causes and effects and although the desire and the impulse was there, it was much too difficult. There is however another school of thought specifically concerned with structures that emerges in the 1930's: *structuralism*.

Structuralism was inspired by a Swiss linguist called Ferdinand de Saussure . Saussure's is the most common understanding of structuralism when the term is used, although I am using it a little more broadly here. Saussure's structuralism held that there were innate structures in language which reflected the underlying reality of the world. These ideas were taken further by an anthropologist called Claude Levi-Strauss who held that similar structures in social systems also reflected underlying reality. This led to a movement that sought essence in the structure of things, rather than in individual elements. A separate school of thought, structural determinism, was defined by the biologist Maturana, in which it was considered that the shape or arrangement of something would determine its future. Structuralism, although quickly forgotten, will go on to influence two other schools of thought with massive ramifications for architecture, systems thinking, and enterprise architecture, which can be described as a very naïve form of structuralism.

What Does This Mean for Software Architecture?

Architects of course have no idea who Saussure was or what structuralism is. What is interesting is that these ideas make their way into architectural thought regardless. How this happens is not entirely clear, but different academic disciplines do seem to cross pollinate. What architecture arrives at is its own form of structuralism, mixing ideas from linguistic structuralism, logical atomism, essentialism, naïve Cybernetics, and Newtonian causality to produce a way of looking at the world that is most readily expressed as structures. I am calling this emergent, unintended structuralism of software architecture *logical structuralism* to distinguish it from other, related schools of thought.

Unlike Russell's logical atomism, logical structuralism is fuzzier and looser. It has no mathematical foundation and seeks to describe structure without ever checking back on its work. It produces structures that routinely fail due to model drift and lacks any form of self-criticism. It borrows promiscuously from other schools of thought with no real coherence, and is passed on through sound-bites, rhetoric, and the lack of any available replacement. Logical structuralism is the default philosophy of software architecture, the culmination of the philosophies of stillness.

It is not surprising that architecture ends up at logical structuralism. The job of the architect is after all to deliver structure. A set of decisions that determine how many components shall exist in a solution, where their boundaries lie, how they communicate with, change and influence each other, how they are physically created and linked to each other. Software architecture is pure structure. From the story I have told so far, it is possible to work backwards, through Saussure, Wittgenstein, Russell, Turing, Descartes, Comte, Aristotle and Plato to understand how we arrived at this way of thinking about the world. It's really quite wonderful to link all of

these ideas together. However, we must also look at the continuing failure of architecture, the software crisis, and ask what role these ideas play in that.

It can be seen fairly clearly in the output of the software architect; the process diagrams, the capability diagrams, the recent business architecture trends that see a business as a set of structures to be mapped in exactly the same way across disparate business contexts. Ideas like requirements engineering and Domain Driven Design place an enormous effort on modeling structures within language to grasp the structure of the underlying business reality. The application of structuralism to social systems is a huge leap of faith, and it is here that we will find model drift and the problems that cause architecture to continually fail. Social structures like businesses and markets are simply not amenable to this kind of structural analysis for the very simple reason that they are never still. Whatever structure may exist is momentary, a snapshot, because the business and the market and the people are in constant movement. The logical structuralism of the software architect is an attempt to hold the real world still so that it can be mapped to software structures.

In the very worst of cases, architecture work has devolved to the kneejerk mapping of these structures, and in doing so become completely detached from the reality of day-to-day business and software engineering. This leads to an architectural practice that is deeply flawed, resented, and ultimately ignored. This is incredibly difficult to grasp and accept because these ideas are being built on exactly the same foundation that made computing possible. Architecture has continued this journey without any real theoretical questioning, and so objectors attack architecture from mostly pragmatic grounds, because it, in the worst cases, works against developers, who don't have this philosophical background to understand why it is failing.

Trying to map the social structures around software has been a standard practice since at least the 1960s. There have been

many subsequent tools that seek to map structure, such as the Zachmann Framework, and even to map the social act of building software. Most architectural tools, such as UML (Unified Modelling Language) and ArchiMate are focused on producing static, generic structural diagrams. These methods are often described with their own process structure, such as RUP (Rational Unified Process). It is not uncommon to find large numbers of these diagrams stored in something called an enterprise architecture repository. Often, they are hopelessly out of date, as the environment they seek to describe has long since moved on, and these documents are often unread and unloved, incredibly vulnerable to model drift and therefore rarely trusted. These attempts at finding essential, repeatable structures that would apply across the industry became known as *waterfall* techniques. Since the late 1990's the agile movement would attempt to remove waterfall thinking from software engineering, rejecting these heavy waterfall processes but failing to recognize or remove the underlying logical structuralism.

Economics, Trade-offs & Neoliberalism

The idea of the trade-off is pervasive in software architecture. Behind this concept is the idea that there are a number of choices that must be made, often in favor of one system property, or universal, over another. This is managed through the use of narratives on utility. If we look at ATAM, a trade-off methodology from 1999, we see that the central technique is called a *utility tree*.

Trade-off is an exercise in utility (a term with its roots in Hume's thinking) that expects a group of stakeholders to make rational choices about which balance between options is correct, or most beneficial.

In the mathematical world, in areas such as aerospace engineering, trade-offs are solved using techniques like weighted sigmoid curves and *multi-attribute utility*. In complex systems there are too many variables to use these mathematical techniques, instead this is weighted by opinions and inevitably softer methods like ATAM¹, which often can give good results, but are more about the management of people than they are mathematical tools. The central concept of utility betrays an adherence to the ideas of standard economics, a Platonic kludge on a complex system that simplifies and distracts. Another concept, that of technical debt, betrays a worldview coming from standard economics, the use of a balance sheet to reason about and drive our understanding of the world. In the mid-2000s enterprise, architecture became enamored with the kind of thinking found in MBA programs, seeing architecture as 'technology strategy', and the ideas of standard economic models and neoliberal thinking snuck in. These models are riddled with Platonic folding. Even the agile movement is seen to have economic influences such as the *laissez faire* approach and belief in the invi-

¹Kazman, R., Klein, M., & Clements, P. (2000). *ATAM: Method for architecture evaluation* (pp. 1-8). Pittsburgh, PA: Carnegie Mellon University, Software Engineering Institute.

ble hand. The mechanistic reduction of the organization to a structure is assumed to have an economic teleology, seeking constituent value, or shareholder returns. In this world, the architects award themselves the task of aligning the technology with these economic intentions. These ideas fit well with logical structuralism, also having their roots in simplistic reductionism, and the combination of the two makes logical structuralism politically compatible with modern management thinking. This unfortunately simply provides another path on which our models can drift.

Modeling Logical Structuralism

All of these ideas in the default philosophy are combined in the belief that they can be captured and represented in a series of models. The roots of model thinking can be found in Newton's groundbreaking work on mechanics, where he very successfully uses a model that approximates reality, using the concept of very small corpuscles that he doesn't believe actually exist to eventually accurately predict the behavior of real-world mechanical systems.

In architecture this is process, capability, object models, all representing the blueprint of the organization. At this point, the architect has entered the Matrix² and created an alternate reality with which to interact. Software architecture will be built against these models and not against the underlying, complex reality. What architecture has succeeded in doing is bypassing the problem of complexity by using these ideas to create models that seem reasonable, based on the beliefs listed in the default philosophy of architecture, logical structuralism. The combination of the *arché* with Descartes' foundationalism, and certainty, the essentialism of Plato and Aristotle, the machine metaphors of Cybernetics, neo-liberal economics and MBA thinking, and the logical structuralism that emerges from this, creates an alternate reality that has the appearance of stillness amid the chaos of the organization, and allows the architect to be confident and certain about design choices. But the statistics on failed software projects tell a different story.

²This is a movie from the 20th century in which the characters live in a simulated reality separated from the real world.

Baudrillard

It is worth mentioning the work of the French philosopher Baudrillard here. As a software person, you will probably be familiar with *The Matrix*, a movie that was inspired by Baudrillard's book *Simulacra and Simulation*. The book even features in one of the film's early scenes. In this book, Baudrillard described a world in which we become completely detached from reality and live through simulacra. These simulacra are dead copies of reality that have no bearing on what is actually there. The approaches of essentialism and structuralism have led us directly to this path. Enterprise architecture has long been there, with the heavy frameworks of the 90s mostly producing maps that have no relation to the reality of the business context.

Summary: The Default Philosophy of Architecture

This invisible, but instantly recognizable, default philosophy makes it possible to accept the engineering paradigm—that software is simply another kind of engineering. By the time the architect has arrived at a set of models created in this way of thinking it becomes obvious that the answers are correct, and the software should be mapped to the models. In traditional engineering, these ideas bring successful projects because the environments are physical, not complex, and the models do not experience drift. These ideas are forced onto software however, and the software crisis unleashed in the 1960s was caused by the fact that these ideas simply did not hold up in real projects.

The perception is that software design *must* be engineering, and the ideas can be fixed by small increments and improvements. Because of this, architecture has become a profession of constant reinvention, with new tools, frameworks and ideas appearing constantly over a period of decades, all trying to right the ship, but never seeming to resolve the central problems of *model drift*.

The only major challenge to this way of thinking has been the agile movement's rejection of up front planning, a clear indication that the kinds of mapping described simply don't work. However agile has chosen to focus on the process of delivering software rather than on the design of software systems, in fact many have promoted versions of agile that reject the need for any up front thinking at all. The default philosophy therefore remains intact, and agile continues to work on models informed by the exact same philosophies of stillness that lead to the default philosophy of architecture. Logical structuralism has become pervasive and invisible, which means it can never really be challenged.

You will find many aspects of logical structuralism in your own thinking, in the thinking of architects around you, in every book

about software design and especially in any book about enterprise architecture. These ideas have come from honest attempts to understand the world around us and are informed by the inherent ideas that led to the development of computer science. This is written not to be critical, but to show that there is another way, that becoming aware of the limitations and origins of this thinking is not *just* skepticism, but a chance to approach the problems of software architecture from a different perspective.

The introduction of a new idea is dangerous. Challenging established points of view can bring a great deal of criticism and exclusion. In 1962 Thomas Kuhn wrote about scientific revolutions and noted that new ideas, those that would cause a paradigm shift, would cause a hostile reaction. Any attempt to change course from this default philosophy would be met with hostility. Kuhn talks about 'normal science' where investigation continues despite mounting anomalies based on the underlying beliefs of the discipline, the current paradigm, as small, cautious increments. Disruptive ideas threaten this peaceful form of investigation and are met with resistance.

The Semmelweis Effect describes the attempts by established ideas and people to try and suffocate new ideas. Semmelweis was a doctor who discovered the effects of hand washing on patient mortality, and rather than being lauded he was excluded and shunned by colleagues at great personal cost to himself. Criticizing object orientation for its naïve Platonic worldview or suggesting that software simply can't evolve in the same way as biological systems, risks invoking the Semmelweis Effect, consciously or unconsciously, in industry elders. Challenging this thinking inside your own company can be fraught with dangers. Proceed with caution!

The major problem with the default philosophy of software architecture is model drift. As soon as the architect believes they have a coherent model, this model informs the software structures that constitute the architecture. As soon as this is deployed (or

even during development), the business context starts to move, and reality drifts from the model. This causes the software to fail, to need patching, and reveals weaknesses in the software architecture. If the model drift is violent enough, then the architecture will fail. Logical structuralism is the problem, it stops the architect from interacting with the world as the world really is and instead engages with the world as how we want it to be. The cure is, unfortunately, worse than the disease.

Walk 8: Skepticism, Rationalism, and Empiricism

At this point you may be feeling a little uncomfortable as a software architect or engineer. Logical structuralism has never been given a name before now, and this default set of beliefs sits so deep in the industry's worldview that challenging it can be very shocking. It is, however, worth questioning these seemingly unquestionable beliefs, an approach in philosophy known as *skepticism*.

Descartes famously doubted everything that his senses told him. He went so far as to not even believe that he existed, because an evil demon could be convincing him of the fact. He arrived at the conclusion that he could not doubt his own thought, since thinking about the demon's possibility was entirely his own doing. He concluded "I think therefore I am". This is often referred to as 'the cogito', as in Latin it is stated "Cogito Ergo Sum". It is about this point that Descartes believes that we can be absolutely certain, and everything else must build upon this initial certainty. This method is called skeptical doubting and could potentially be very useful for the software architect who has many evil demons trying to sell them variations of stillness.

A century or so later, Scottish philosopher David Hume also developed a project that involved skeptical doubt. What Hume doubted, however, was philosophy itself. He believed that philosophy had fallen into a sorry state and relied on acts of imagination and speculation with no recourse to evidence or scientific method. Hume believed that we could know nothing except what we learned from experience—*empiricism*—and rejected Descartes' *rationalism* — a

belief that knowledge and certainty comes from reason and innate ideas. When you hear agile enthusiasts talk about empiricism, this is what they mean—a belief that knowledge can only come from experience. Hume called the entire scientific project into question, denying that there could be any scientific certainty. He put forth some radical ideas, such that there was no such thing as the self, but that we were merely bundles of sensations. His ideas were *critical*, aiming his skepticism at accepted ideas, often referring to them as superstition, and he was incredibly critical of metaphysics.

Hume's empiricism caused some issues for rationalist thinkers on the continent. This was during the Enlightenment and Hume's powerful arguments seemed to lead to determinism for the mind as well as the material world, suggesting that free will did not exist at all, that we were merely following a trajectory already set in motion.

All of these approaches, at their core, share that same search for stillness. Even though Hume and Descartes arrived at skepticism from very different routes, they both centered their thinking around the ability to be certain and to be free of uncertainty. Hume's thinking is often grouped with other British empiricists such as Locke and Berkeley, who laid the groundwork for modern, empirical, scientific thinking, building on the work of Francis Bacon in the 1500's, and Hume's ideas will be foundational for the logical positivists discussed in the first Walk.

We have already discussed two of the major ideas Hume contributed to this journey. The first is that causality is not something we can see or experience. All we experience is that two things seem to happen in constant connection to each other. The other is that our way of deriving knowledge about the world through repeated, similar experiences, known as *induction*, is a fairly shaky approach to knowledge, since it cannot be relied upon to be true, and cannot be shown to hold in the future.

Hume's skepticism was often criticized, in fact skepticism is some-

times seen as a very negative thing within philosophy, and even more so in software. Skepticism can be seen as a negative philosophy, a way of dismissing but never arriving at anything new. But sometimes, as we shall see, burning everything down is the only way to move forward¹. Socrates was also skeptical, pushing back on people's definitions, revealing their ignorance, to the point where he got himself killed. Skepticism can be controversial. In the situation where the software industry finds itself, however, skepticism can also be a powerful tool for navigating the avalanches of ideas that are produced every year. Skepticism in the style of Hume will be used to great effect in a later Walk to help clear the way for a new philosophy of architecture, which is exactly what Hume was trying to do for philosophy in general.

The ideas of the rationalists such as Descartes and Spinoza can be contrasted with the empiricists such as Hume, Locke, and Berkeley. Both sides of this argument practiced skepticism to be rid of baggage which had accumulated. In truth, most rationalists like Descartes have empirical beliefs and approaches, and no empiricist is free from using logic and deduction. The lines are much more blurry than simple definitions allow.

The discussion of the differences between these two schools of thought can help us understand how and why we think about problems in architecture. If we allow some blurring of these definitions, the architect might employ a more rationalist approach, and the agile software developer a more empiricist approach. The truth is that both forms of knowledge are almost always present, but there seems to be something else at play in the work of the architect.

Rationalism in software architecture might be likened to design work. Before actually building software, we need to think about its optimal structure, and this requires intuition, followed by deduction, reason and logic. A rationalist might argue that our ability to

¹"May the bridges I burn light my way" –Dylan McKay/Luke Perry, Beverly Hills 90210. A lesser known work of philosophy from the 1990's.

abstract (which is essential to design work) is intuitive and innate to us as human beings (or *a priori* knowledge).

A more empirical approach would be to build some software, expose it to reality, and learn from that experience, or *a posteriori* knowledge. The empiricist would believe that the rationalist simply cannot perform their duties without experience, and the rationalist would believe that there are underlying, innate intuitions that allow us to address things without direct experience of them.

Let's take the empiricist position that the architect's ability to reason about a system design is only possible because of prior experience of previous projects or education on the subject. This will lead to a strange paradox—if knowledge is gained through experience, how can an architect solve a problem they have never seen before? The natural counterargument in response to this paradox would be to say that the architect is extrapolating from previous experience, which would imply that there must be some kind of shared essence of software architecture across business contexts that can be reapplied to a new project. The software crisis would suggest that things are not as simple. Attempts to capture this supposed shared essence have resulted in the development of ideas like software patterns and reusable software structures, reference architectures, that can be applied across many different business contexts. This has also led to attempts to define essential architectural processes that are the same across business contexts. Neither of these approaches seem to work, and, in most cases, these generalizing impulses require enormous effort and additional work in order to be successful. This is most apparent in the constant surprises and escalating costs of large, one-size-fits-all corporate ERP(Enterprise Resource Planning) projects. Customers are essentially buying model drift as a service!

Successful architects must develop a set of skills that allows them to diverge from previous experience and create something new in new contexts. A rationalist take according to which architects possess some innate ability to do this, doesn't seem to hold since

you will rarely meet architects who do not have several years of experience. There is a rationalist intuition at play, but it doesn't appear to be innate. I will argue in later chapters that successful, senior, architects develop a sense of how to *doubt their experience*, a set of heuristics that can create new lines of thought through the use of intuition and imagination. This doubting can be employed to make the conceptual models of the architect stronger.

Skepticism in Software

Skepticism is a great skill to practice when designing software systems. For a start, being skeptical that you can bring to bear the stillness of Platonic or analytical thinking to the meandering, half-baked ideas of business stakeholders would help a great deal.

The discussion on rationalism and empiricism has many implications for architecture. Is it possible to build an architecture a priori—without the experience of building the software and watching it fail, or is it possible to build without any up front planning or thinking and use the experience of this to drive design? Do we even have to choose a side? There are many in the industry with strong opinions on this, and many who seemingly succeed from both positions.

Now that I've talked about the default philosophy of software architecture, and introduced the idea of skepticism, and discussed rationalism and empiricism, you might be wondering where I am going to take you next. If skepticism is to produce anything useful, apart from mere doubt, it must produce something new. In the case of the default philosophy of architecture, it would be nice if our skepticism could produce a new philosophy of architecture, free from the issues of the default philosophy. This involves understanding the influence that Plato, Aristotle, Descartes, Russell, Frege, and Wittgenstein have had on our thinking, tearing this down, and asking—is there another way?

SECTION II: The World That Refuses to Be Still

Interlude

The skepticism of Hume plays an important role in the history of philosophy. It led to the awakening of a giant of philosophy, Immanuel Kant, who would influence thinking in the West through to the present day. This gave rise to a swathe of diverse thinkers who begin to question the philosophies of stillness and propose new ways to see the world, actively questioning the logocentrism of previous approaches. This rise will lead to splits, opposition, confusion, and ultimately aporia.

In the software industry this same phenomenon can be seen in the rise of the agile movement in the late 90s, with the publication of The Agile Manifesto in 2001. This work and the movement that followed questioned the heavy, rigid approaches of software engineering up to this point. The Agile Movement, just like the philosophers in this next section, became spread out, with different groups advocating different approaches and eventually ending in aporia, with the agile movement today questioning its own purpose and many approaches now resembling the rigid approaches they once opposed. The questioning of the philosophies of stillness and the questioning of rigid approaches to software have a lot in common.

Architecture & Metaphysics

At this point, we need to introduce an important term: metaphysics. A simple definition of *metaphysics* is the study of everything that is outside of the physical realm. Metaphysics according to Deleuze—a French philosopher from the last century whose mission appears to have been to write texts so impenetrable no one would ever finish them—is the study of the difference between essence and appearance. The term was made popular because of the collective name given to some of Aristotle’s works, *Metaphysics*, which in Ancient Greek simply meant “After Physics”, as his previous work had been called “Physics”. Defining metaphysics is hard (and controversial), but discussions about universals, ideal forms, and being vs becoming all belong to metaphysics. There are many hard problems in philosophy, and naming things is definitely one of them, a curse philosophers share with software engineers.

Metaphysics is often accused of being speculative and nonsensical, and many philosophers have tried to get rid of it altogether, end it, or solve it. None have really succeeded. The logical positivists especially hated metaphysics and wanted it gone, in favor of a more down-to-earth, scientific version of philosophy. Metaphysics is, by its very nature, rife with speculation. The positivists wanted everything to be proven, verified before it could be accepted, with empirical evidence gathered from experience and experiment.

Positivism is obviously not the only school of thought available to think about the world. For instance, a different way of thinking, born in the social sciences, is called *interpretivism*. For an interpretivist, knowledge is gained not through discovery by experiment, but through ideas and debate. Knowledge about social systems cannot be gained by running the same kind of experiments as in the empirical sciences.

On reflection, it becomes obvious that, as software architects, we have been taught to see the world in a positivistic way, when in

fact, we learn in an interpretivist way—through debate, and through listening to ideas at conferences. We don’t run experiments, nor do we expect absolute clarity in the logic of these ideas, nor do we even empirically verify them. We have been caught in a trap of behaving as interpretivists while believing we were scientific—once again, *belief without knowledge*—sophistry.

Metaphysics is a natural part of the software architect’s work. The ways in which we interpret the world around us is always done through some kind of belief system, under which a huge number of assumptions are collected, sometimes hidden from view. When the original ideas of the positivists fail us, we fill the gap, as Kant mentioned, with whatever is lying around. For centuries this “whatever” was the idea of divinity, nowadays it is something like consumerism, and for the last few decades in software architecture it has been agile, or SOA, or microservices, or whatever the latest trend happens to be.

Aristotle’s original metaphysics was very much focused on stillness; on first causes, on being, over time the definition of metaphysics has changed as this assumption of stillness was challenged.

Henri Bergson, who we’ll meet in a later chapter, believed that science itself was missing a metaphysics. This statement can also be seen to be true for the discipline of software architecture, and a problem we should have at least attempted to have solved by the end of this book.

Walk 9: Being and Becoming.

So far, this book has mostly focused on knowledge. In philosophy the study of knowledge is referred to as epistemology. Another branch of philosophy, ontology, is concerned with the nature of reality, asking what the world is. Two competing ideas in ontology are the notion of being, things which exist, and becoming, things which are constantly changing.

There is a still ongoing debate in Western thought around the issues of being and becoming. Being is focused on Socrates' favorite question "What is it?", and becoming is focused on the question "How did this come to be?" These two questions differ greatly in the assumptions they make about the world. If one believes in being as the primary question to be investigated, it will be necessary to sideline becoming as an illusion, or as a distraction, and focus on the things that can be apprehended, that are right in front of us. If one focuses on becoming, it seems obvious that there is nothing static in front of us to investigate at all, everything is changing, all the time, and being, as a static unchanging thing, is the illusion.

Parmenides, who focused heavily on being, is often contrasted with another pre-Socratic philosopher, Heraclitus, who focused on the question of becoming. Heraclitus believed that the world was in constant chaotic flux. He is famous for the phrase 'No man steps in the same river twice, for he is not the same man, and it is not the same river'. This paints the picture of a world in which everything is moving and constantly changing, like the flowing of water in a river, or the ideas and attitudes that make up a person. Heraclitus' world of flux is familiar at once to the story of software architecture. In this world, the model drift, which, in the last Walk,

we saw how much effort we mobilize to stop, is not a bug, but a feature. In Plato's *Theaetetus*, Socrates rejects Heraclitus' world of constant flux. Plato, as mentioned, sought stillness and saw it as the rightful state of things. Parmenides, and then Plato, put a great deal of focus on Mathematics. For Plato, Math was the highest possible skill, and the heroes of Plato's dialogues, like Theaetetus often revert to mathematics. Mathematics is pure reason, pure logic, pure thought, undisturbed by our sensations, pure in its form and incorruptible. A triangle will always have 180 degrees, regardless who is contemplating it or when. This tendency to elevate mathematics above other forms of thought continued through time right up to von Neumann, Turing and Hoare and the founding of computing. Many of us who start careers as software engineers are attracted to this world of mathematical purity, where uncertainty is obliterated by proof and logic, and nonsense is banished to the void. Plato was influenced by Pythagoras, a mathematician, the leader of a bizarre cult who believed in the primacy and divinity of numbers, an idea to which Plato was open. There is a connection between the belief in the divinity of mathematics, and the belief in stillness and logic. A link from Plato all the way to the logical positivists. Russell, writing about universals, states:

"The world of being is unchangeable, rigid, exact, delightful to the mathematician, the logician, the builder of metaphysical systems, and all who love perfection more than life."

When we think about why software architecture is hard, we are in fact retracing the arguments of the ancient Greeks, who saw that there was certainty in some things, and uncertainty in others. Some, like Heraclitus, want to embrace that uncertainty, some, like Parmenides, wished to be rid of it. In the early years of software engineering, the urge was undeniably to be rid of uncertainty, from Turing's attempt to prove programs correct before they run, to the rise of formal methods, to the creation of enterprise architecture, with its rigid structures. The pushback came first in the form of the agile manifesto, and now, as you the reader will come to see,

in the later Walks of this book, as we take some philosophical steps beyond the agile manifesto and start to really wrestle with the drift between our beliefs and our knowledge.

Being and becoming represent two opposed positions that allow us to view the world very differently. This dichotomy colors much of the history of philosophy and is a good lens through which to view the twists and changes of Western thought. And, as you will see, it is also an excellent way to express the problems we experience in software architecture.

Walk 10: Kant, Hegel, and How We Think About Things

Hume's skeptical philosophy tells us that we can't really be sure of anything, even ourselves, referring to our notion of self as just an incoherent bundle of perceptions. By Hume's reckoning, the assumptions we as architects make about the business context, in terms of induction and causality, can never be shown to be true. This means that just because a certain process works today, it may not work tomorrow, and if a certain event provokes a response in the market a few times it may not do so next time. If you observe a few software projects, you will find that this is very much the case, and if you read Hume too deeply as a software engineer you might just fall into despondence and give up—it can never be correct! As I pointed out in an earlier Walk, should Hume have been right about his fork, there is no way for us to ever be able to validate our architectures, given that they can't be analytically proven, and empirical testing is always too late. Kant offers us one way out through an investigation of how we think and relate to objects in the world.

Given that in Walk 1 I described the architect's job as creating conceptual structures that could be mapped to code, how an architect creates concepts is incredibly important. How do we take in information from the world around us and produce those concepts?

In a paper written in 2016, Ralph and Tempero¹ interview a large

¹Ralph, P., & Tempero, E. (2016, June). Characteristics of decision-making during coding. In *Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering* (pp. 1-10).

number of software engineers and ask them how they arrive at their decisions about code structure. The answer is overwhelmingly that they do not know. Architects seem to be out of touch with how they arrive at their decisions, how we arrive at the conceptual models that define our architectures.

A software architect will immediately, following the default philosophy, start to break down the business context into essences and structures. The boundaries for these essences will often follow some kind of verb-noun analysis of stakeholder descriptions. This will lead to the identification of essences like Customer, Order etc. The next step will be to identify relations between these essences—either through shared properties or through functional affects that group them together. These essences will then be mapped to objects and to relational or data storage structures, or the functions to services, and this starts to form the conceptual model. This thinking is a mix of logical structuralism and whatever arché the architect is currently thinking with. This is most often entirely instinctive, it is rarely explicit and rarely understood even by the architect themselves.

The problems that the architect will encounter now are well known. “New” requirements will constantly appear. The Customer and Order essences will constantly be updated as new information is discovered. Experienced architects will realize that the object-oriented software developer will create different essences than the relational database developer or the business analyst. In fact, the mismatch between these things has been a great source of problems over the years. The architect will quickly discover that the essence of Customer is a very different thing across different departments and partners, and also that it will change over time. Eventually the organization will need to engage in mapping and validation projects to pass Customer entities around.

How did we learn to do this? Where did we learn to break business contexts down in this way? What makes us think that we are doing this correctly? Given the problems that software architecture

experiences with model drift, how do we have any confidence in these structures? Despite the constant failures caused by model drift, every project attempts to solve these problems with exactly the same thinking—a common definition of madness.

The idea that we can break things down in this way of course comes from Russell and Wittgenstein's view of logical atomism. The idea that there is an essence is Platonic. These approaches have been shown to be very flaky indeed over the years. The interesting question the architect must ask is how we arrive at these decisions; how do we make sense of the world around us? What exactly is a conceptual model, inside our heads? How do we get from stakeholder stories to data and code structures?

As software architects we are used to working with generalizations and essences. This is primarily because software exists to make business operations more efficient, it seeks to automate as much as possible, and the very act of automating means that we are seeking generalizations that can be repeatedly executed in the same way and that can be solved through computation. How do we know what these essences are? Do we just assume their existence? How do we know what a customer is? Isn't a purchase event, a mixture of customer and order, a better way to think about the business context? Does our choice of technology influence our view of the world? And how do we explain the constant changes that our concepts (and therefore code) undergo during an application's life cycle? Is it the case that we don't actually know what a customer is? Is it the case then, that we are stupid? Or that it simply isn't possible to know what a customer is, for some reason? German Idealism, a school of thought that emerges in response to Hume's skepticism, produces some ideas that allow us to look at these things from a different perspective, and in doing so, gives us some clues about why we think the way we do, and what limitations we are working under.

One thing that we can quickly observe is that other roles, such as business analysts, with no training in computer science, can

equally well create these concepts in the business domain. Is there a shared knowledge that allows us and them to do this? We have very different educational and work backgrounds, so it is not a function of pure experience. Knowledge gained from experience is referred to as *a posteriori*. If not from experience, is it possible that there is some kind of innate understanding that all humans have that allows us to form these concepts, that this is something outside of computer science? This kind of knowledge is called *a priori* knowledge. Is our ability to create these concepts learned or innate? Another path to knowledge is through logical inference, for example, being able to say that *all customers have names*, where the knowledge is contained in the definition of customer. This is referred to as an analytic statement. This is opposed to a synthetic statement, where we could say that *all customers on average visit our store twice a year*. It is only possible to make synthetic statements based on a posteriori knowledge, says both Hume and common sense. The ability to create these concepts is central to the architect's skill set, and we don't seem to have any idea about how we do this.

How do philosophers describe this process of concept creation? An enormously simple way of thinking about concept creation is to believe that we simply observe the world, that the world makes an impression on us, which allows us to create concepts that describe exactly what exists in the real world. This is known as *direct realism* and almost no one believes it to be true. Alternatively, it is believed that we are limited by our senses in how we perceive the world around us. What we experience are appearances. A dog, or a fruit fly, given their differences in biology and eye structures, will not experience things visually as we do. Already in the work of Descartes we see the realization that we may not have full access to what is actually there. Hume takes this even further. Hume introduced the idea that the reason that essences and appearances could never be reconciled was because of us, because of the limited way in which we see the world. There could never be scientific

certainty. A predecessor of Hume, George Berkeley, believed that there was nothing outside of our minds at all, a position known as idealism, or solipsism. In between the naïve world of direct realism and Berkeley's idealism, lies Kant, who believes that some of the world is constructed by us, some of it is real, and what is real, cannot be entirely accessed by us. Kant believes that we interact with objects in the world not directly, but through a representation of the object that we *cognize* Kant takes this confusing question about cognition and gives the readers of this book what they really want: an architecture! Kant attempts to create an architecture of our cognitive faculties to explain how we come to know things about the world around us.

Immanuel Kant's most famous works were inspired by the skepticism of Hume. He started as a rationalist and claimed that the empiricist Hume had awoken him from his dogmatic slumber. Kant's aim was to prove, as Hume seemed to deny, that we had free will. Kant was a supporter of the enlightenment and individual freedom. Hume's deterministic philosophy, which left no room for human freedom in a determinist universe, greatly disturbed Kant. Kant's philosophy proposes a third route, neither rationalism nor empiricism, giving us as subjects a central role in creating the world around us. He set out to create a philosophy that would show that humans could have free will. He adds some prongs to Hume's fork, one of which is controversial and considered by Hume to be impossible: the existence of *synthetic a priori* knowledge. That is, that we can innately know some things without experience that aren't analytically obvious. Kant achieves this by focusing on how we think, how we come to understand the world. Kant agrees with Hume that we gain much of our knowledge from experience, but that experience is processed and ordered by pre-existing structures in our minds, synthetic a priori knowledge, that we are born with. Kant believes that our senses limit our ability to know objects(noumena) in the world, and instead we create representations(phenomena) and interact only with these.

The objects of our experience, thought Kant, were a product of our interaction with them, but our representation of them was different than the things themselves, what Kant called the *ding-an-sich* (*thing-in-itself*). Our cognitive faculties and senses limit the possibilities of our experience, and this limitation means that we create the conditions for experience a priori. Three examples; space, time, and causality, are considered by Kant to be invented by or pre-existing in our minds, to order the experiences that we have, implying that time, space, and causality may not even exist without us. We process the world around us using these innate ideas and structures such as categories and schemata, to make judgements about the world around us. This was called Kant's Copernican Revolution. Copernicus caused a bit of a stir when he suggested that the world revolved around the sun, instead of the sun revolving around the world. Kant was now suggesting that instead of us revolving around the objects of the world, as Descartes would have it, they were revolving around us, as we actually created the conditions for representing them. These ideas are referred to as *transcendental idealism*.

Reason, according to Kant, is how we organize these experiences and create higher level concepts. Kant believes that our reason doesn't really know its limits, and there are always things that cannot be proven, but we will still try. This leads to paradoxes Kant describes as *antinomies*, such as the existence or non-existence of God. These cause circular arguments where neither argument can be proven. Kant recommends that we take a position here based on faith, there is the possibility of free will, the possibility of God, which cannot be ruled out or be proven, and we can choose to believe in them. Kant sees this as both a bug and a feature. Our rationality is then by Kant's measure destined to lead to these antinomies - waterfall to agile, monolith to microservice, strongly typed to dynamic languages, should architects code or not code? These antinomies are often experienced by architects in a way that is often described as analysis paralysis, a very tiring kind of aporia.

Kant's philosophy is highly speculative and will be keenly debated, renewed, updated, and rejected over time. However, these aspects of a priori knowledge, antinomies, and faith, are all relevant discussions for today's architects who are deeply disconnected from their own internal thought processes and don't understand how they arrive at the conceptual models they create.

Kant's idea that we are responsible for ordering the world around us according to our own rational and cognitive limitations is fundamental and fascinating. We have synthetic a priori knowledge of the world because we actually create it! As architects we can easily see that architects have no innate sense of how to create these models, but that they create their conceptual models around something that they have chosen to believe in, in this case not God, but instead things like capability mapping, object orientation or event sourcing, and these articles of faith go on to influence the conceptual models we create. A similar Copernican revolution must happen within software architecture where we realize that these ideas are not out there being analyzed by us, but internally and dramatically shaping how we see the world.

Kant is a philosopher of stillness, a bridge to a new understanding. He is a philosopher of stillness because he believes his architecture of cognition is still, fixed, and already existing. To really make Kant's ideas relevant to modern software architects, let's fast forward a little to the 20th century and see how amazingly relevant a less static picture of these ideas is. Kant is guilty of the enterprise architecture mistake of assuming the existence of a generalizing top down model, never realizing that individuals are perhaps creating representations in entirely different ways, based on what they chose to have faith in. This seems a little contradictory given Kant's belief in the individual freedom of the enlightenment.

Relative a priori

Kant's thinking would greatly influence the ideas of the logical positivists who would go on to be crucial to the development of computer science. In 1920 Hans Reichenbach wrote about an idea that is very important in the discussion of architecture—*relative a priori*. This is the idea that what is a priori is constantly changing because of experience, with the example being holding the beliefs of Newton and then changing them as we learned the ideas of Einstein, how our conceptions change over time. This is an important idea for architects, because our relative a priori is constantly changing, and the arché that each architect brings with them affects this relative a priori, which is updated after each and every project, lecture, and debate. This has the effect of making every developer and architect have their own relative a priori beliefs, while Kant in his original form informs the enterprise architect's belief in a singular, centralized, fixed a priori view of the world, setting developers and architects on a collision course.

As an architect, each time you go into a project, your a priori beliefs are updated, you are starting from a different position as you gather experience. Since software architecture has no real scientific foundation, we are constantly creating it. This is an important step away from the philosophy of stillness.

The default philosophy of architecture, logical structuralism, describes a large chunk of the default a priori view of most software architects. This is fluid, however, updated by the individual architect's views and experiences, embracing one or more arché. This means that if you embrace object oriented design, or a particular pattern without question, you approach design with this as part of your a priori knowledge, before you even begin to experience the actual problem to be solved, you are already shaping and forming the world around you, much like Kant believed we do with time, space, and causality. By adopting a particular idea, architects begin to order the world to support that idea. Most of us with a few years

of experience in architecture will have met many architects and developers who work in this, often frustrating, limited way.

This tendency is dangerous for architects since we so readily believe in our technologies. The void that is left by the absence of a metaphysics of software architecture means that we are all alone, forced to create our own worldview based on the limited experience in our own careers, framed by the platforms we work with.

That we are responsible for how we order and view the world is perhaps the most important lesson we can take from Kant, and once we've appreciated that we should never accept the default philosophy or take on other's ideas lightly, as this greatly impacts the conceptual models we are capable of building.

It is vital for any architect building conceptual models to understand Kant's perspective. The idea that we structure the world around us, that our reason has limits, and that we must take leaps of faith should all be a part of how we think about our own thinking as architects. That we are limited by our own knowledge in how we build models is a key thought here and would lead to architects seeking as much experience of the world as possible for their conceptual models to be less limited. Our reason has limits, and if we don't dial it back, we'll end up in aporia.

Kant's work inspired a great deal of argument and philosophical development. He inspired the school of German Idealism which is a rich and varied collection of ideas and thinkers. A number of thinkers set out to either show that Kant was wrong, or to build upon his ideas. Jacobi sought to show that Kant was wrong, Reinhold to improve Kant's ideas. Fichte built upon Kant's work, focusing more on the subject of how we come to understand ourselves. Schelling provided many interesting arguments, sometimes not entirely coherently, but many that support the direction we will take in later Walks. Schopenhauer built upon Kant's ideas to derive a new way of looking at the world, but it was their contemporary, Hegel, who would claim to complete Kant's work.

Hegel offers a view of knowledge as ever evolving, as becoming, as a movement toward an end. In doing this, Hegel describes a world not of still things, but of moving ideas and a reality in which everything is interconnected. A world much more familiar to the software architect.

Hegel brings back becoming

Hegel would build upon Kant's ideas and produce a way of thinking that shattered the previous attempts at stillness, that explained that our way of understanding the world, was inherently unstable, chaotic, and not at all still. Hegel promised of course that at the end of this chaos there would be stillness, something he called the Absolute, the end goal of human development. His ideas really do form a "system", a complete description of everything in the world. His works are enormous, difficult, and are subject to multiple, often contradictory, interpretations. Hegel's way of thinking is referred to as *absolute idealism*. The absolute is perfect, self contained knowledge, in which there is no distinction between ideas and things, they are one.

Absolute idealism brings forth two important ideas for our discussion of software architecture. The first is that everything is interconnected. Rather than focusing on our relationship to individual objects in the world, as Kant had done, Hegel saw the world as one, with our thoughts and judgements as part of that one, all interconnected, all moving. This shift in perspective will turn out to be enormously helpful to any philosophy of software architecture, helping to rid us of the habit of separating the software from the business world in which it executes.

The second important idea is that everything is *becoming*. Hegel brought the idea of becoming back into the spotlight. Hegel described how our knowledge grows through constant contradiction, with every idea containing its own contradiction that over time becomes unavoidable. This leads to a constant process of *sublation*², or creation, of new ideas, and this cycle continues until an end in which we would come to know God through the *absolute* (*there are different interpretations of this*). Hegel was a historicist, who looked to history to extrapolate the future, assuming that the

²The German word is *aufheben*. For some reason, all writings about Hegel include this. I include it here because I'm afraid someone will shout at me if I don't.

world would continue to unfold as it had in the past. Hegel believed that we were close to reaching the absolute, that all of the world's history had been moving toward this end, and that the Prussian state represented the ideal that would help get us there. The entire premise of this book, that philosophy has unfolded from a search for stillness, to confusion and meaninglessness, toward a philosophy of movement, and that software architecture is on the same path, is a *little* bit Hegelian.

At the center of Hegel's thought is an important idea— that our knowledge, our concepts, our essences are necessarily incomplete and are always on their way somewhere else—they are always *becoming*. Hegel did not separate reality from thought, and saw them as the same thing, with rationality as the motor driving the development of concepts. Hegel was an idealist with some rationalist traits. For him, any difference between essence and appearance was merely confusion that would eventually be resolved through his own version of dialectic.

Hegel was considered the last of the great systematic philosophers, producing a system which tried to explain everything, like TOGAF or SAFe or RUP. Fundamental to Hegel's thought is the idea of how a concept becomes, or comes into being. He believed that every concept or idea contained the seeds of its own destruction, a negation, something that would make the concept invalid. This is often described as thesis-antithesis-synthesis, a description actually written by Fichte, but often attributed to Hegel. In this movement, a concept survives until the negation becomes obvious, at which point a new concept is created, and the two are fused into one, with one *sublating* the other. This continues over time, until we reach the absolute, or absolute knowledge, a path on which we have already been set. Hegel thus brings back teleology, and the idea that the entire world is set on a path toward a purpose or an end.

It is possible to view the progress of the IT industry through a Hegelian lens. The constant proposal of new ideas, each time found

to contain their own contradictions. The swings from monoliths to microservices and then back is reminiscent of the Hegelian dialectic, and this is a different way of understanding this than Kant's antinomies. The only disappointment is that we don't seem to have reached the "Absolute" in software engineering, in fact, our pendulums just keep swinging. Hegel is different than Kant in believing that rationality will eventually deliver absolute knowledge, knowledge of everything, instead of antinomies and the separation of appearances and real things.

Hegel's contribution to our journey as architects lies in how he saw the creation of concepts as a continual process, with everything connected to everything else. Rather than the remote Kantian subject observing the world, Hegel saw the subject as part of the world, and the development of concepts happening amid a constant flux. Our conceptual model then, in Hegelian terms, is going to undergo constant change, and this is indeed what every architect working on any non-simple application, will experience as they try to understand the business context. Hegel gives us hope that the constant churn we experience when developing our conceptual models and code isn't our own incompetence, but it simply is how things are meant to be.

Walk 11: Reversing Platonism and the Crisis of Meaning

Some of the most fundamental questions in philosophy boil down to: what is the world? What are we? Are we just a bunch of stuff related to other stuff, solid and unchanging, with our poor little minds overwhelmed and confusing this overwhelm with change? Or are we living in a world of constant flux and change where the real illusion is that anything stands still at all? For millennia philosophers have pondered these questions and there have been some trends and shifts, but the influence of Plato remains strong in the Western world, and his still, eternal, ideal forms dominate our thoughts and actions even today. If you are a software engineer these ideas might seem esoteric, but they underlie the very same thinking you use when you design software. The way you think about the world will be a mixture of the ideas that we have talked about so far. The ideal, unmoving forms of Plato, the certainty and realism of Descartes and the rigidity of logical positivism combine to form a worldview excellently prepared to tame silicon chips and woefully inadequate to deal with the world of people, business, and social organizations.

After Hegel, a new kind of philosopher appears on the scene. Kierkegaard, a Danish philosopher, turns inward toward the self. Nietzsche declares that God is dead, that life could easily become meaningless, and that science threatens us with nihilism—a lack of belief in anything. Kant talked about how we filled the void, the phenomenal gap, with whatever was lying around, with faith, and Nietzsche was worried about what was actually lying

around, afraid that society would descend into nihilism, believing in nothing. Nietzsche was openly frustrated with the state of his society, viewing it as holding back the individual from greatness. He railed against Christianity, morality, and platonic thinking, vowing to reverse Platonism. His style was difficult and he refused the systematic philosophies of Kant and Hegel. Heidegger would eventually also question Plato, and Greek philosophy in general, claiming that we had neglected the very basic question of what it meant *to be*. Heidegger, a generation after Nietzsche, would claim that the act of philosophy in the Western tradition was getting in the way of our understanding of ourselves and our place in the world, and eventually he would accuse technology of doing the same thing. In “The Question Concerning Technology” Heidegger described the way in which we allow technology to get between us and how we experience and understand the world and referred to this as *enframing*. Heidegger rejected essentialism and indeed much of the existing metaphysics, and his writings focus on how traditional philosophical thought led to the exclusion of context around us—a description of the central problem I am discussing in this book. He wrote “logic is only one interpretation of the essence of thought” and presented many alternative ways in which we exist in and interact with the world—rejecting for the most part the *subject-object distinction* so important to Kant and Descartes. He thought that we exist in the world as an inseparable part of it, not as a separate subject observing objects.

Heidegger developed the idea of the Hermeneutic Circle, in which we reconcile our everyday experience of things with a broader understanding of the whole, building up a picture over time. In this way, we interpret new information constantly in the light of what we previously knew. Heidegger’s work, is interesting because of the rejection of essentialism and the embrace of context, that meaning is giving to a thing by its surroundings, by preconceived notions, by experiences over time, and that it changes. I have heard that the Hermeneutic Circle has been taught to undergraduate

software engineers as an example of how to interface with social systems, and this is encouraging!

Heidegger's rejection of logic led of course to ridicule by the logical positivists, and his obscure writing style and invention of new terms didn't make his philosophy easier to defend from their attacks. He built his ideas on the work of Edmund Husserl, who invented a new branch of philosophy called *phenomenology*. Husserl saw phenomenology as a science of consciousness, and he was interested in how we experience the world as conscious beings. His work focused on something called *eidetic reduction*, which was a form of essentialism seeking to find the essence of specific human experiences, but also their becoming. To do this, Husserl described a method known as bracketing, in which we close out everything else to focus on the particular experience we are investigating- this is very similar to Dijkstra's separation of concerns and Heidegger rejects it for much the same reasons - it isn't possible to disconnect one thing from everything else around it.

These ideas led to a search for meaning, culminating in the well known work of Jean Paul Sartre. This represented an inward turn, a rejection of Platonic philosophy. Sartre claimed that *existence preceded essence*, that essence was a distraction. Sartre's existentialism rejected the previous philosophies replacing them only with a sense of meaninglessness and absurdity, and offered little in the way of solutions to anything. The existentialists simply embraced the uncertainty of the world.

An existentialist philosophy of software architecture would look like the following; a rejection of the old Platonic ways, and an embrace of the meaninglessness of it all, working without any theory, believing in nothing, but seeking to find meaning and define that meaning for ourselves. It is up to every individual to find their own meaning. The agile movement, in many ways, was an existentialist embrace of software engineering. Many of the topics of the agile movement were not empirical ideas that would definitely work, but instead, very often a statement of how

developers wanted to work, what was meaningful for them. The way that I teach architecture is not far from this, encouraging architects to find their own path, tools, and to think independently and contextually.

Some of these ideas would help to break down the Platonic ideas a little—opening the door for a better understanding of becoming.

Nietzsche's most important idea, for the purposes of this book, was the contrast of the Apollonian and the Dionysian. The Dionysian was wild and uncontrolled, the Apollonian logical and ordered. He believed that the Apollonian and the Dionysian were both necessary to understand the world, that the banishment of poetry in favor of science would be a disaster for mankind. This is the world in which the software architect finds themselves—caught in the swirls of the unpredictable business environment, but responsible for the logical, cold execution of software. Nietzsche realized that the two depended on each other, a lesson not heeded by software engineering at all.

Nietzsche declared that the onus was on every individual to define their own life, that it could not be defined socially, that there was no philosopher king, but a “superhuman”, or *übermensch*, which we should strive to be. His writing is negative and harsh, but many have found something positive and inspiring in it.

One of Nietzsche's most famous ideas was the will to power, the idea that there is some kind of driving force behind our actions pushing us forward. Schopenhauer influenced Nietzsche here with his work “The World as Will and Representation” where Schopenhauer talks about the driving force behind why people do what they do. A few decades later Henri Bergson defined ‘*élan vital*’, a life force that drags us forward.

These three ideas all echo something important, that goes back as far as Aristotle's unmoved mover, the original cause at the root of all causes, that something is moving, that there is movement, rather than there being something still. While Platonism had sought to

paint the world as still, divine and unchanging, Schopenhauer and Nietzsche had seen that there was something within us that wasn't still at all, and that seemed to them to be driving us. At the same time as these debates moved forward toward Bergson's *élan vital*, Darwin published *The Origin of the Species*, which suggested that things were indeed not still but in fact subject to evolution. While Nietzsche's rage sought to undo Platonism, Darwin's discovery was a much more convincing case that stillness was not something that could form the basis of our natural world.

Darwin's ideas on evolution at last broke the spell of essentialism—the thinking of biologists changed dramatically from one of categorization to one of understanding how species changed over time.

With these thinkers the overriding philosophies of stillness were suddenly challenged, and it is precisely here that the philosophy of architecture finds itself today.

Sartre's search for meaning, Camus' embrace of absurdity, Nietzsche's rage against everything around him, and the fear of nihilism, can all be seen in the social media feeds of software developers today, very often borrowing Nietzsche's most famous trick and declaring something to be dead at least once a week. Software architecture now finds itself in a twilight zone, at once trying to shrug off the philosophies of stillness, the default philosophy, the old waterfall approaches and the new, restrictive, contradictory agile frameworks, and yet having nothing substantial to replace it with. The result is cynicism, mockery, withdrawnness, and a sense of absurdity.

Software moved from its period of seeking stillness to a similar period of questioning, cynicism, and rejection as philosophy. The agile movement attempted to shift the focus of the industry from the stillness and certainty of what was termed *waterfall*. In many cases developers who ascribed to agile methodologies and ideas were, like Nietzsche, Heidegger, and Sartre, seeking meaning in their work, emphasizing their own experiences, and questioning

the rigidity of absurd processes that promised logical consistency but did not deliver working software. The agile movement is today considered troubled and is quite often declared dead with the same drama with which Nietzsche declared God to be dead. Just like in philosophy, this period produced many diverse thinkers, new ideas, and sought to replace the old with the new, breaking established truths and seeking meaning, identity, and new directions. While agile started to grow, enterprise architecture strangely became even more rigid and steadfast and also started to grow. These two schools of thought, the stillness of enterprise architecture, and the rage and movement of the agile movement, were both in direct opposition to each other and yet flourished at the same time. This quite strangely mirrors the major philosophical divisions of the 20th century—often described as analytic and continental philosophy.

Analytic vs Continental Philosophy

In Walk 2, I described the problem of certainty and uncertainty in software architecture. In the 20th century, philosophy would take two very distinct paths in dealing with this problem. This split is central to the theme of this entire book. The architect finds themselves stretched between two worlds. Over the course of their computer science education, the focus is on logic, algorithms, reductionism, and analysis. Once the architect starts to practice in the real world, they will realize that the uncertain and illogical world of social systems are very different and that the tools they have learned won't bring any resolution to the problem. The two major, often opposed, schools of thought that dominated the 20th century, known as analytic and continental philosophy, almost seem to mirror the internal struggles of the software architect—one focused on logic, rigor, and scientific approaches, the other, viewed through the analytic lens, is fanciful, flighty, dramatic, and nonsensical—not unlike the picture of business stakeholders a lot of software engineers develop when trying to grasp the flux of business environments. Hegel, Nietzsche, Heidegger, and the existentialists would eventually come to be seen as *continental philosophy*, contrasted and compared to the analytical school of Frege, Russell, and Wittgenstein.

The one important idea that I do want you to understand at this point is that there is a huge problem in software architecture that these ideas might help us to fix. The enormous difference in the worlds of software and the world of human organization and business context is the greatest difficulty we face, and we are expected to face it without theory, without training, and with tools only developed for the software side of the problem. This obvious split in software architecture is reflected in the split of continental and analytic philosophy, and this is worth digging into!

SECTION III: The Philosophies of Movement

Walk 12: Philosophies of Movement

In this walk I will introduce three schools of thought that can be considered philosophies of movement. That is, philosophies that admit the lack of stillness, that hark back to Heraclitus and the world of becoming, and that ultimately give us as architects some tools with which to start building a new philosophy of software architecture that moves away from abstract, imaginary stillness and toward the movement and flux that we experience in our everyday work. In this section I will be focusing a lot more on the so-called continental philosophers.

Process Philosophy

Process philosophy is a way of thinking that sees the world as being made up of constantly moving processes, it is a world of becoming, rather than being. Its most famous modern proponent is Alfred North Whitehead, the collaborator of Russell on the *Principia Mathematica*. His seminal 1929 work *Process and Reality* places process and becoming at the center of his thinking. Whitehead sees the world as being made up of events. The things we relate to every day are simply collections of these events.

Whitehead was in good company, his thinking was inspired by Henri Bergson, who was famously the subject of Russell's ire, and who, before Whitehead, also saw the world in terms of becoming. Bergson defined *multiplicity* (borrowing from Riemann, the mathematician). A *qualitative multiplicity* is something that has many heterogeneous things contained within it, but it cannot be separated into or reduced to those constituent parts. (From now on, when I use the word multiplicity, I mean qualitative multiplicity). This multiplicity is not something that can be reduced, planned, mapped, or even predicted. Multiplicity is an important concept here because it is a description of all the things that do not make themselves amenable to either logical analysis, essentialism or structuralism, and it is not deterministic. When we look through the ideas of the philosophers we've talked about in this book, Bergson's multiplicity is the first concept that comes close to describing the architect's experience of a business context, and also something unreachable by the Platonic and analytic schools of thought. Bergson's critique of the philosophies of stillness is, for me, one of the most important philosophical developments from the perspective of the software architect.

Bergson provoked the ire of many physicists, including Einstein, by suggesting that scientific time was simply a trick to make it easier to reason about things. His concept of *durée* questioned the scientific account of time, separating time from space and referring to *durée*

as a qualitative multiplicity, better approached with a method he called *intuition* than analytic methods. In doing this, Bergson is suggesting that time is not simply reducible to a number of points, moments, on a line, in the same way we treat space. Here he is making the same argument against reductionism that this entire book is making. This made Russell extremely upset, especially as Bergson became one of the most famous philosophers in the world, people flocked to his lectures, and he even caused traffic jams in New York on his first visit to the United States.

Durée is a succession of events, all merging into each other so that they seem irreducible, something that is not reducible to parts, but is an interconnected bunch of stuff—again what Bergson calls a *qualitative multiplicity*. This duration is heterogenous, (having many different elements), and indivisible. All the things the architect experiences in the business domain are not simply sets of things interacting with each other, they collectively are durations, measured not by the ticking of clock, but by their relationship to each other, with their states melting into each other in a way that can be sensed but never analyzed. Whitehead refers to this irreducible motion as actual occurrences, a constant drip of events that creates the world around us. Bergson suggests the method of *intuition* as the way to approach duration a way of avoiding the reflex toward analysis, reduction, essentialism, that involves contemplating the entire duration. A great example he gives of this is of a melody. Each note on its own is meaningless, it is the succession of notes, their phrasing, how they bleed into each other, that forms the melody, and the melody must be experienced as one by the listener, not broken down and analyzed note by note. This experiencing of the melody as a whole is done by intuition. For the architect, the movements in the business domain are the melody, and we have been taught to identify single notes, hierarchical diagrams, value streams, or process descriptions. Our stakeholders can sing the tune, but we can never hear them because of our instinct to analyze and reduce. Logical structuralism can

be likened to the effect of earplugs—we know there’s a tune, but we can’t quite hear it because we’re watching the piano and the pianist’s hand movements instead of listening.

Bergson’s intuition is the closest any philosopher has come to describing what the software architect must do to reconcile the business system with the rigid software system. Bergson’s intuition involves resisting the urge to reduce, to spatialize, to immediately start looking for concepts upon which to base our understanding. Fans of Bergson will describe his intuition as rigorous, but as an engineer you will find it anything but rigorous in the sense that we use the word. However, find your favorite senior architect and ask them how they view a project from the outset, and you’ll hear something that chimes with Bergson’s intuition; resisting the urge to analyze too soon, looking wider than the immediate problem. A common problem for readers of Bergson is that they see people mix up Bergson’s intuition with every day, gut-feeling intuition. The problem with accomplished senior architects is the opposite, that they often mix up their natural gravitation toward Bergsonian intuition with everyday gut-feeling intuition—and so when asked for an explanation of how they arrive at a design, the answer will often be “gut-feeling”. For the architect schooled in computer science with its links to the verificationism of analytic philosophy, this admission often comes with a sense of shame, of having failed.

Bergson describes a philosophy of movement in which our ability to analyze fast moving, multivariate systems is limited, but our ability to experience, process, and understand them is still there. As people we are able to manage social situations simply by living through them, and attempting to analyze them would be strange.

Bergson believes that these antinomies, and thus those we experience in software architecture by extension, are the result of this tendency to break something down and analyze its constituent parts. The arguments about certain frameworks pitted against others are unnecessary, it is the attempt at reductionism that makes the arguments appear. Bergson also refutes Kant and believes that

we can have absolute knowledge of the things in the world, through intuition rather than analysis.

Both Whitehead and Bergson describe the world in terms of the same flux as Heraclitus, and this approach to philosophy is known as *process philosophy*. The ideas of Aristotle, through to Descartes, and Kant are collectively known as substance philosophy, and these ideas do not help when working with emergent phenomena, in fact they get in the way, and they are the mainstay of most approaches to software architecture.

The concept of process philosophy can be seen as uniting the ideas of the existentialists, the post-structuralists, and Nietzsche, and the anti-platonic thinkers. Instead of seeing a world as consisting of things relative to a human mind, we see the human minds as part of world of processes, indivisible, unstoppable, and in constant flux.

Post-structuralism

For software architecture, the pressure to be able to describe the flux of the business world through the logical lens of computing ensured the focus would fall on the already outdated and outpaced structuralism as a way of thinking. Structuralist thought, as I explained in Walk 7, pervades the industry and academic literature on software engineering. The positivist training of computer scientists made structuralism a more familiar home for architectural thinking. The logical structuralism that I am so critical of in this book is one specific to software architecture, that arises from the original structuralist ideas, the logical atomism of Russell, determinism, and reductionism, and the early forms and structures of computers and computer programs. This is something which architecture desperately needs to break free from. The post-structuralists present a chance to break free, but it requires reading some very hard and frankly obscure philosophy.

In 1966 a philosopher called Jacques Derrida travelled to the USA to give a speech on structuralism and shocked the audience by delivering a highly charged anti-structuralist speech. This event gave way to a new group of thinkers, sometimes (unwillingly) called post-structuralists, who sought to define the world in an anti-Platonic way, moving ever further away from essences, and beginning to question the reality of everything analytic philosophy held dear. These thinkers included Michel Foucault, Jacques Derrida, Althusser, and Gilles Deleuze. Many of them had started out as structuralists but came to reject that way of thinking. In many ways, the very quick move from structuralism to post-structuralism represented a microcosm of the slow move from essentialism to Darwinism that had occurred over the last two millennia.

Post-structuralism is often derided as nonsense, an example that made it clear to the positivists that engaging with nonsense outside of the scientific method was sure to lead to nowhere. Central to post-structural thought was Derrida's *deconstruction*, a ques-

tioning of what lay behind a text, that led to a questioning of ideals and structures that existed. Derrida's concept of *différance*—where the meaning of a word is under constant change, but never really reaches any defined meaning—could be easily describing the precarious position of any software object or model in a world that moves faster than we can update software.

Post-structuralism is a polarizing and difficult set of ideas to process. It is a very different set of ideas, almost completely opposed to the positivist way of thinking. The analytic philosopher Searle referred to it as 'Fashionable Nonsense'. The texts are heavy and hard to follow, and have many references to literature. At first glance there is nothing that is obviously relevant to software architecture, as the thinking is meant to free us from structure, and software architecture is pure structure. Reading this material as a software architect, it is clear that you are very, very far from home.

In *Difference and Repetition*, Deleuze fuses the thinking of Bergson, Nietzsche, Spinoza, and Hume to produce a radical new take on how we see the world. Instead of Hegel's constant negation, where concepts develop through their own internal contradictions, Deleuze claims that we simply experience the same thing many times, repetition, and the most important aspect of that repetition is the differences between each repetition that we notice, which allows us to build up concepts over many repetitions.

Deleuze argues that there is no essence in a changing world. In fact, the only essence that exists in our exploration of the world is that every repetitive exploration gives rise to different ideas than before. This is a true reversal of Platonism, where the essence is in fact the flux! Deleuze argues that the focus on essence, on identity, has obscured the world from us. The only thing that objects in the world have in common is that they will be different every time they are contemplated. This is an incredibly powerful idea. This idea will turn out to be incredibly useful to the software architect as we shall see in a later walk.

Deleuze's philosophy was a rejection of the substance philosophy of Aristotle, as well as of Platonic idealism and he even tried to distance his work from Hegel, although some made the case that he protested a little too much and that he actually owed a huge debt to Hegel's rediscovery of becoming. Deleuze's philosophy is a clear example of a *process philosophy*.

Bergson's star faded quickly, and it is Deleuze that is often credited with bringing him back into philosophical focus with the book *Bergsonism* in 1966. Deleuze sees duration as something that is defined by difference. This difference also cannot be represented, only experienced. It is this difference that makes the world knowable. Deleuze borrows heavily from Bergson, and the idea that emerges is that of *difference through repetition*. As we repeatedly encounter something, intuition alerts us to differences, things that cause the model we have of the system to break. We then rebuild the model with new understanding. There is no essence here, as it keeps breaking, it is never still enough for long enough to be considered an essence. This is similar to Hegel's dialectic, but there is no teleology and no focus on negation, just difference. As a software architect, Deleuze's Difference and Repetition is my favorite idea in all of the philosophy I have studied.

Pragmatism

One of the biggest American contributions to philosophy is known as Pragmatism. A way of thinking that values the outcomes of our modes of thought more than introspective reflection about what they might mean. One of the most common responses to my lectures on the philosophy of architecture is a pragmatic one: what should we do with this information? What is the outcome of this philosophical approach?

I think it is important to include the pragmatists in this section for two reasons. Firstly, because they have a genuine philosophy of movement which ultimately rests on the impact that an idea will have on the real world, and secondly because the concept of pragmatism is widely misunderstood in the world of software engineering. Any architect trying to slow down and contemplate the problem will inevitably hear calls to be more ‘pragmatic’, to simply get something done. This impulse makes pragmatism the second most Googled philosophy in software engineering after Wittgenstein. Pragmatism and acting without thinking deeply stand in direct contradiction of each other!

The main pragmatists were Dewey, James, and Pierce (pronounced Purse to keep pedantic philosophy undergrads off your back). Pierce defines something called Tychism—the idea that the world is made up of pockets of order floating on disorder; echoing Nietzsche’s Apollonian/Dionysian distinction and the workday of the software architect. In the philosophy of the pragmatist, we also find the grip of Plato loosening. Pierce defines a very important concept: fallibilism. Fallibilism is the idea that none of our beliefs should ever be considered conclusive, that any of them could be false. In the shifting of business contexts, it is important to remember this!

The work of James is also a philosophy of movement. James, a psychologist, is keen to preserve the fluidity of thought. He was

an admirer of Bergson's work and the two have much in common. James is known for an idea he called *radical empiricism*, that the world can never be adequately interrupted in order to perform an analysis, it cannot be held still. He formed a pragmatic theory of truth, judging an idea not specifically for its correspondence to reality, or its coherence with other ideas, but instead the practical implications of its use.

In the field of software architecture this pragmatic theory of truth is extremely important. Does it actually work? Is the most important question we will be faced with when creating a software structure.

Induction, Deduction, and Abduction.

One of Pierce's most interesting ideas is that of *abduction*, as *opposed to induction or deduction*. Induction is the way we generalize about the future by using the events of the past. Hume famously pointed out the problems with this - simply because something has happened a particular way in the past does not guarantee that it will happen that way forever. Popper expounds upon this problem and it is still debated today. The way in which we form hypotheses about the world is very shaky indeed, and is never guaranteed to hold over time.

Deduction is the use of logic to move from one statement to the next by logical inference. The most common explanation is of the syllogism, defined by Aristotle.

Abduction is an attempt by Pierce (Purse, remember!) to describe how we come up with our hypotheses, or new ideas. It is a kind of side stepping of logic. The truth is that we don't actually know where our ideas come from. This prong was missing from Hume's Fork. How do we as architects arrive at our decisions about the structures of software applications? Given that there is so much variation across contexts, we must use abduction to propose hypotheses of how a particular software structure will survive in a certain context. The software architect's work is therefore a set of hypotheses. William James in "The Will to Believe" makes the case for belief in hypotheses before seeing evidence, in order to be able to investigate them.

Summary

Process philosophy gives us another way of describing the world as becoming, as the essentialist, structuralist, view of the world is suddenly not the only show in town. These thinkers and ideas are undoubtedly the hardest to grasp. They are seeking ways of explaining things that are simply not expressed earlier, and some of them attempt to do this by inventing new language, which makes their work even harder to understand.

An easier way to relate to the ideas covered in this walk is to think about how we as architects make decisions. Think about the component breakdown in your latest project and how you made decisions around it. Try to describe the process by which you arrived at these decisions. The truthful answer, for most software architects, is that this mechanism is nothing more than a gut feeling. In a business context in which things are poorly defined, stakeholders are themselves confused, and the future is unknowable, the software architect resorts to gut feeling to make their decisions. Our kneejerk response to this realization is often one of disappointment—the positivist in us is disappointed in our lack of rigor. This is not mathematics. And yet, for senior architects, it works! There is an excellent book by Donald A Schön, *The Reflective Practitioner*¹, that talks about the ability of professionals working in complex domains to, after some experience and reflection, be able to make decisions that are good, but not logically defensible, they are based on sense and intuition built upon reflection over long careers. Our understanding comes from something that isn't logical analysis. This is what the work of the existentialists, and of Bergson, and of Pierce, are trying to describe.

Instead of near-decomposability, reduction of a system to a few interacting parts, or the description of a structure that explains the future, these thinkers introduce concepts that speak to the

¹Schön, D. A. (2017). *The reflective practitioner: How professionals think in action*. Routledge.

architect's experience of their own intuition. Bergson's multiplicity is the most important. An inseparable, indivisible entity, that seems to be driven by some sort of invisible force, an *élan vital*, is much closer to the reality of a business than the structural diagrams, org charts and logical analysis of stakeholder language. Deleuze's *Difference and Repetition* or Bergson's intuition provides a better account of how we learn in these situations than anything in the positivist or structuralist canon.

The 20th-Century swirl of Pragmatism, Existentialism, process philosophy, and post-structuralism can all seem like very different takes on a non-Platonic world, but they are all addressing the same problem—if the world is so logical, reducible, why can't we simply explain it? Just like in Parnas' 1986 article, "A Rational Design Process, How and Why to Fake It"², we've been faking it. Software architecture is trapped in a world of behaving one way and pretending to behave in another. We believe ourselves to be rigorous, no-nonsense logicians filtering nonsense to produce correct software, but we learn like interpretivists, through debate, through the sharing of ideas, through taking different relativist positions. The ideas of Deleuze and Bergson sympathize with our position, but they do not tell us how to solve the problem. To do this, a pragmatic approach will be needed where the ideas are judged on their real-world outcomes rather than their internal coherence.

This is of course, easily dismissed as nonsense because it isn't easily explained, it makes no logical sense, and gut feeling can't easily be taught. This goes some way to explaining the apprentice model of learning in software architecture, since we have been completely incapable of teaching these things.

²Parnas, D. L., & Clements, P. C. (1986). A rational design process: How and why to fake it. *IEEE transactions on software engineering*, (2), 251-257.

Evolution

A word that has crept into the lexicon of software architecture is the word ‘evolve’. A good architecture should “evolve” alongside the business context as it changes. This is an attempt to align architectural thinking with the philosophies of movement and as such is the right instinct. Architectures, however, do not evolve, as evolution is something that happens to species, not to individuals. It might be possible to say that cloud systems evolve, after testing products across thousands of different customers some features die and new ones appear in the system, but this is an analogy. The idea of evolution as some kind of directed, designed, adaptive process is not what Darwin meant. Instead, architectures simply change. They are not random, they do not have mutation, and the use of evolution as a metaphor for architecture is severely misplaced. Architectures are challenged and then changed with intention. Software, if left in a forest running on a machine with solar panels will still be running in exactly the same way in 1000 years, when you come back to it. Compare this to what will happen if you leave a few bunny rabbits in the forest and come back 1000 years later. There is a general confusion about the concept of evolution, that it somehow is intentional, that a species somehow wants to survive, or intentionally mutates toward a goal. Giraffes did not stretch their necks because the trees got taller, the short ones simply died and produced no more short offspring. Of course, there is a desire for software to evolve, to learn from its environment, but evolution is a messy, lossy, expensive process whose cost, complexities, and inefficiencies make the software crisis look like a picnic. Selling software that evolves is a desire, not a real-world solution, and a philosophically flawed idea. Individual projects do not follow the same trajectory as a species with random mutations, and to state that they do is either wishful thinking or willful pseudo-science.

We use the vernacular ‘evolve’ to mean changes in response to movements in an environment. When the river wears a stone to

a mere pebble, the stone has not evolved, because there is nothing internal to the stone that drives this change. Darwin's evolution cannot be extended to stones, or to software. The reason that we talk about this wishful evolution is because one of the serious difficulties of designing software is that the environment around the software, the business context, is in a state of constant flux. Humans have survived in a fluctuating environment, and therefore it makes sense to try and build software that mimics our own evolution, especially when the Platonic efforts fail to keep up with the changes in the environment.

It would appear that software that evolved would solve all our problems. Sadly, this is not something that is mathematically possible. In a later Walk I will talk about the work of Stuart Kauffman, who will show in his book "At Home in the Universe" that the mutation and evolution of software to solve an actual problem couldn't possibly happen before the heat death of the universe. If we are going to produce a philosophy of software architecture that can cope with movement in the business environment, we need to stop comparing software changes with biological evolution almost immediately. To replace the philosophies of stillness will require more than mere analogies to anything that seems to move.

Software has been on a journey since the first programs were written. Some of our ideas, such as the movement from procedural thinking to objects, to services, have seen ideas change over time, and many times you can trace the Hegelian dialectic, as we move from monolith, forced by the negation, or failure of the idea, to microservices, and then swing back, with the ideas constantly evolving and changing. This is absolutely not the same as the biological evolution of Darwin.

Software systems, instead of evolving in the Darwinian sense, have a tendency to move forward in the sense of the Deleuzian Difference and Repetition. This is closer to Hegel's dialectic than Darwin's natural selection, because there is the intentionality of our thought behind the process, and it is about a shifting idea,

where the software follows the idea, rather than a direct relationship to its environment.

Darwin's ideas had a transformational effect on the thinking of the scientific world. Suddenly, things became much less linear. Aristotelian essentialism had been one of the main driving forces in biology up to that point, and suddenly it seemed much less important to identify species and categorize them, and much more important to understand how they came to be, and where they might be going.

Walk 13: Emergentism

In the twilight of 20th-century philosophy, the positivist project has been rejected, and structuralism and post-structuralism had failed to achieve any breakthrough. A new school of thought, however, had been brewing for a long time.

The idea that ran through the thinking of the determinists like Descartes was that a system could be broken down into its constituent parts. For the structuralists, it was the relationships between the constituent parts that formed the essence. Everything in the system could be explained as a result of the behavior of these constituent parts or their resultant structures.

Despite the many successes of Newton's ideas, there were still very many areas of knowledge that remained inaccessible. These areas failed to be adequately addressed by positivistic, reductionist, or scientific methods. Areas like economics, psychology, sociology tried to adopt scientific methods but the result of this is today often referred to as the replication crisis. These experiments and results are unfortunately not replicable in a large number of cases. A new idea is needed.

Emergentism

In the 1920's, a number of philosophers began to posit the idea that new properties could emerge out of something that were not contained in its parts. That there was something more. The germs of this idea can be traced far back to thinkers like Aristotle and especially John Stuart Mill in 1843.

The first and most understandable example of emergence is consciousness: consciousness is something more than merely electro-chemical reactions. By gathering physical entities together, neurons, something has emerged that previously had not existed, a mind with a sense of itself. It cannot be seen, touched, predicted or controlled, measured, or broken down into its constituent parts, but it is definitely there and is real.

A student of Russell at Cambridge, C. D. Broad, was one of the first to talk about this idea in greater detail. In his book *The Mind and its Place in Nature*, he described how mental states emerge from a sufficiently complex system. He describes an emergent property as something that is impossible to predict from the arrangement of its parts.

In his 1925 book, Broad discussed two opposing theories—mechanism, and emergentism, and notes that we cannot actually know what is emergent and what can be reduced. For our purposes, mechanism can be applied to the world of software, and emergentism to the business contexts in which software must execute.

Emergentism is often divided into two schools of thought. Weak emergentism, and strong emergentism. In strong emergentism, the emergent properties of the system are not present in the individual elements, they are something entirely new. It is this strong emergentism that is interesting for us software architects. When the emergent part of the system affects the constituent parts, causing the system to change over time, this is known as *supervenience*.

The presence of an emergent system, together with supervenience, causes constant motion as the emergent system pushes on the parts, and the moving parts cause changes in the emergent system. This is reminiscent of the *élan vital* of Bergson, the will of Schopenhauer and Nietzsche, the invisible hand of Adam Smith. This creates systems that are perpetually in motion, always becoming.

The notion of emergent systems poses some challenges. The philosophies of Aristotle and Plato become difficult to apply as there is no essence in emergent systems. They are constantly shifting and changing, so there is no essence to grasp onto. The idea of causality disappears, as does constant conjunction, as the same inputs give different outputs across time. Without static universals to work with, Russell's logic cannot be applied. In the emergent system, the ideas of many of the philosophers who contributed so much to the eventual invention of the modern computer, the philosophies of stillness, flail around helplessly.

This is not an unknown problem. Plato's dialogues, when they touched on what we would now call emergent and supervenient systems, such as justice, knowledge, or piety, ended in *aporia*. Wittgenstein's approach was to pass over these things in silence, focusing only on what could be addressed by logic, which by this stage in our journey is a vanishingly small part of our world.

Another example of real world emergentism is a toothache. First, we ask the question "*What is it?*" As Socrates would have asked. Is a toothache a thing? The instinctive answer is no, a toothache is not a thing, because you can't touch it, you can't see it, you can't measure it and you can't divide it up into smaller parts. But it is undeniably *real*. A toothache definitely exists, if you ask anyone who has experienced it. You might make the claim that a toothache is a state of affairs, a collection of properties of several things happening at the same time that lasts for a certain amount of time. If this length of time is short, we might reference it as an event. If we allow this, then what is a human being? Is it not simply a collection of properties, existing for a short period of

time? The difference is that a human can be touched, seen, exists in the world. The other difference is that a human will exist for a longer period of time than a toothache. The difference between them is one of duration—the longer something exists, the more substantial it becomes, the more likely we are to say that it exists. Thus temporality, or at least endurance, is incredibly important in establishing what is going on around us.

The toothache could be considered to be a universal. In that case, what is it that every toothache has in common? What is it that unites all toothaches? Russell makes the argument that universals must be real in some sense because if they weren't no two people could experience them. Aristotle would claim that the toothache is the same thing across different people. However, how do we know that each of us experiences toothache in the same way? Emergentism allows the toothaches of the world to exist, to emerge, and to be different from each other. That roughly the same phenomena occur over and over again does not mean that they are the same, that they share anything. In an emergent world the universal is nothing more than a compression, a way of storing information efficiently, and not necessarily a metaphysical relationship between every single toothache. In making this statement I am subscribing to nominalism, that these universals only exist in our language.

This is interesting when we consider the concept of agility in software development. Agility, like a toothache, is not something that can be touched or seen. However, it is definitely experienced by many developers. The assumption is that agility is a universal, that it is the same across projects and organizations, and that it can be brought about by observing cause-effect relationships and replicating them. In truth, agility is emergent, it is the product of millions of different variables, all moving and changing, and is impossible to describe with simple causality. Thus it cannot be replicated, mapped, or caused, but when it emerges it feels fantastic, as anyone who has worked in truly agile teams where change is fun rather than a problem, will contest. This of course means that agile

or agility do not exist outside of the teams it emerges in.

In this world there are of course still things. These are often seen as the result of processes moving slowly. Whitehead refers to them as eternal objects, a rather controversial position. Substance philosophy flounders when it encounters these emergent systems, and process philosophy flounders when talking about things. This is the split between essence and appearance, and the same idea appears multiple times across the history of Western thought; the Apollonian/Dionysian of Nietzsche, *natura naturata/natura naturans* of Spinoza, Cartesian dualism, a problem that runs to the heart of philosophy and appears many times in these different guises. Many philosophers flounder on this boundary, and it is across this boundary that the software architect must work every day, the boundary between the static software and the emergent business context. The most pragmatic approach is that of Descartes, to simply separate the emergent from the static and focus on the static that can easily be accessed. This has been the default of software architecture, choosing to focus on static software patterns instead of the flux of the business system.

It should be fairly obvious by now why so many thinkers focused on stillness. They were trying to keep the emergent system still and were constantly caught out by its movement. A philosophy of substance must keep the emergent still to address it, which leads to failure and disappointment, because this stillness is a desire, not a fact of the world. Just as Kant described, we could not prove that the stillness was or wasn't there, but it was possible, and so we chose to believe in it anyway. The exact same problem has been experienced by software architecture across the last few decades.

How is it possible for us to live our lives in these emergent systems then? Our biology, our social relationships, our economy, our society are all emergent. The truth is that it is perfectly possible to live without a complete logical understanding of these systems. We learn to live in them through experience, through constant exposure. Hegel's dialectic allows us to begin to grasp this, the

constant changing of concepts as we contemplate the systems we live in. Bergson's intuition tells us to stop trying to analyze, spatialize, or reduce these things.

A business context is like an emergent system. It cannot be reduced to its parts, or even fully understood. It is best approached by intuition, not in the everyday sense of the word, where we mean gut feeling, but by something closer to Bergson's intuition. Mapping, modelling, and reducing will simply produce artifacts that get in the way of our understanding, as described by Baudrillard, or Nassim Taleb's *Platonic Fold*¹, or Heidegger's *enframing*. The reason I can list so many examples of the same thing is because this problem has been recurring in philosophy for a very long time, and it is centre stage in software architecture. Trying to reduce this to a series of snapshots, or an essence, is impossible. Positivists like Russell have laughed off notions like Bergson's intuition, but how else do you navigate social systems, structures, or systems that keep moving despite your attempts to keep them still? It is not by reduction, logic, linguistic analysis, by mapping or orchestrating, or predicting. Our imaginations become useful in discovering possible futures in these systems which we cannot know the future of, and analysis shuts these down. Things like poetry, music, art, storytelling are acts of imagination that can be seen to play a role in navigating, teaching, and learning how to work with emergent systems. In this world, logic and mathematics will simply reduce our paths to understanding, they will obscure more than they can reveal, and this is the message of philosophers like Nietzsche and Heidegger. Intuition and imagination are powerful tools in this environment. Many of the architect's tools pose as scientific and rigorous mappings of reality, in truth they are nothing more than vehicles for storytelling in the business context, closer to poetry than code or mathematics. Remember that in the *Republic*, Plato wants to ban poetry. Deeply logical and serious software engineers will recoil at the idea that their work is closer to poetry than

¹Nicholas, N. (2008). The black swan: the impact of the highly improbable. *Journal of the Management Training Institut*, 36(3), 56.

mathematics. However, emergentism describes things that are simply not addressable by the mechanistic philosophies that have dominated computing. The architect's paradox leads to us being so focused on the notes that we can't hear the tune.

The way that architects have learned is closer to Schön's reflective practice and Bergson's intuition than a mathematics textbook. Clichéd and unimaginative descriptions of architecture allude to a "mixture of art and science", believing themselves to have said something profound, and this is what they are alluding to. However, just because intuition seems unscientific doesn't mean that we cannot produce something scientific from these ideas, we simply have a different starting point.

The first lesson I try to teach to becoming architects is to stop analyzing, to simply relax and let the details of the business context buzz around them, to enter the business context and experience it without thinking about arché, patterns, requirements or risks. My first naïve readings of Bergson's intuition made it seem like he had figured this out 100 years ahead of time.

Now we have arrived at a set of philosophical concepts that allow us to describe the environment we work with as software architects, and we understand the roots of the ideas that we had before. Now there are two questions to be answered; if this isn't just speculative philosophy, then we should be able to show scientifically that this is how the world works, and secondly, what will this look like for software architecture?

Emergentism as Science

These ideas can be seen in a different light if we draw lessons from the complexity sciences. Take a biological system like a human body, or a weather system like a tornado, or a mechanical system like a pendulum with a two-part leg, or a chemical reaction, or a bunch of teenagers. All these things seem to have a universal property, from our perspective. When we try to understand them, we get very confused. They seem to have patterns but then suddenly do something unexpected. This makes us upset. We spend time mapping out their structures, categorizing them, seeking essence and structure, and then before we even finish the job, they change. This is annoying. They are durations, multiplicities, with strong emergence and supervenience. Is there a scientific way to approach this problem? Can we, somehow, arrive at empirical evidence for something that actually works?

Walk 14: Complexity Science

In this walk I will fuse the ideas of emergentism, complexity science, and Bergson's and Deleuze's thought. Bergson and Deleuze bring philosophies that seem to describe the experience of the software architect working with emergent business contexts. I believe that both were describing what we nowadays refer to as the science of complex systems. The ideas of Bergson and Deleuze, the pragmatists, and emergentism are, as we have seen, very abstract and it will perhaps be difficult to convince software engineers to spend a great deal of time on these. However, all of these ideas are talking about the same thing; irreducible, unpredictable, systems that are difficult or impossible to analyze by reduction. In the latter half of the 20th century these problems would have a new discipline dedicated to them: the complexity sciences. It is here that I have spent the last ten years. It is important to understand complexity from a philosophical perspective in order to make it useful for architecture. Without the lessons of Bergson and Deleuze it is very easy for the software architect to take the scientific view of complexity and snap back to direct analysis, reducing and essentialising. It is entirely possible to restate the problems in the language of the complexity sciences, or in logic, or in mathematical terms, and sound very clever whilst doing so, but it adds nothing to the discussion. There is, unfortunately, no essential complexity that ties together all the systems that we struggle to understand, and a general understanding of complexity will not help you solve the problems of software architecture. Essentialism and structuralism, as we have already seen, are a huge cause of the problems of software architecture and the last thing we need is more essentialism, logical structuralism, in disguise trying to rescue

us.

Although the ideas of Cybernetics and systems thinking touch around the edges of what will eventually be called complexity science, the most significant work in the complexity sciences arrived later, not from a general approach to complexity but from a genuine exploration both of dissipative systems in Prigogine and of biological origins by Kauffman. These are scientific works, but also, in my opinion, proper works of philosophy because they produce new concepts. Prigogine describes dissipative systems as complex, disordered systems that should fall toward stillness and equilibrium, but maintain some kind of internal order through being open to energy inputs from outside the system. Kauffman works to explain the order inherent in biological forms and how they come to emerge from chaotic systems.

These complexity sciences proper leave essentialism behind and deal with emergentism in a scientific way. The work of Kauffman describes the presence of *attractors* (a concept borrowed from Lorenz) in complex systems, states which the system returns to again and again. (A more detailed description of these can be found in my first book¹, my academic papers², and Kauffman's own work³). Alicia Juarerro⁴ refers the constraints that elements place on each other in a complex system and how these constraint effects are a type of complex causality.

The philosopher Manuel de Landa⁵ takes the idea of the attractor from Kauffman and describes the impact of such a concept in philosophical terms, giving it a much broader reach.

Whilst the essence is a fairly natural way to work in the world of

¹O'Reilly BM, Residues: Time,. Change and Uncertainty in Software Architecture, LeanPub, 2024

²O'Reilly, B. M. (2022). Residuality Theory, random simulation, and attractor networks. *Procedia Computer Science*, 201, 639-645.

³Kauffman, S. A. (1995). *At home in the universe: The search for laws of self-organization and complexity*. Oxford University Press, USA.

⁴Juarrero, A. (2000). Dynamics in action: Intentional behavior as a complex system. *Emergence*, 2(2), 24-57.

⁵DeLanda, M. (2013). *Intensive science and virtual philosophy*. Bloomsbury Publishing.

substance, of things, essence becomes impossible to work with in emergent systems. The attractor becomes the closest thing to an essence when discussing emergent systems, a state that the system returns to again and again. Every emergent system can be seen as a collection of attractors.

The concept of the attractor, alongside the concepts of supervenience and emergentism, at once make it easier to understand speculative concepts like *élan vital*, why a hermeneutic circle is better than identifying individual things, and why difference and repetition lead to a better knowledge of an emergent system than the seeking of abstract forms or categorization. It explains why true understanding or meaning are always slightly out of reach, but tantalizingly close as in Derrida's *différance*. Suddenly, the speculative metaphysics of the last century seems to have been stretching toward a reasonable description of what is now considered scientific phenomena, if somewhat scattered and confused. The ideas of post-structuralism and process philosophy turn out to be quite useful in describing complex systems⁶, and were thus a little bit ahead of their time.

The attractor replaces the *essence* and the *universal* — it becomes the primary thing that the architect must identify to create a conceptual model. That which appears universal is a mirage, and although sometimes that mirage is a helpful shortcut, it often impedes the ability of an architecture to survive in its environment. For example, instead of defining a universal called “security”, which gives the impression that there is something called security which is the same across different business contexts, it is better to first describe the attractors that a business system may arrive in. The thing that is most interesting then, when considering the problem of model drift in software, is the *differences* between these attractors, how security of the application is affected in different situations in its own environment and context, instead of focusing on what

⁶Cilliers, P. (2002). *Complexity and postmodernism: Understanding complex systems*. routledge.

they have in common with other contexts. The attractor allows the architect to escape the trap of immediately defining universals or forms, while still compressing the impossibly large business context. The result of this kind of thinking is that we dissolve the traditional boundaries of non-functional requirements, dealing with the kinds of attractor the system will be exposed to directly, instead of trying to squeeze in universal concepts like “security”, “maintainability”, or “performance”, with accompanying universal toolsets and thinking, and then trying to optimize for the particular context. This will give a true picture of what is actually necessary. This is terrible news for those in the industry who sell products or frameworks that fetishize a particular non-functional aspect of applications and rely on the existence of universals across contexts. Naïve essences like Customer or Order must be adapted to survive across these identified attractors and the shifts between them, either absorbing differences or splitting into multiple entities.

The consensus in management complexity today is that the way to solve problems in complex systems is through experimentation and incremental actions, building up knowledge as we run experiments. This is an empiricist position that mirrors the scientific method of analysis, and, as such, will find few who argue against it, given the positivistic background of computer science. However, the way in which we traditionally have emerged victorious from battles with complexity—that which is scientifically beyond us—is through the creation of new concepts. The examples in my first book still hold—Darwin’s finches, Newton’s apple, Copernicus’s revolution, all were inspired, lateral, leaps of imagination not at all driven by the scientific method—but by abduction! Instead, they made the scientific method possible in a different way by affecting the design of the experiments and the generation of hypotheses. The agile movement suffers greatly from the delusion that experimentation and iteration are enough, paying little heed to the concepts that drive the design of the experiments or the underlying belief systems that define how they believe the world actually works. If these

are wrong, experimentation delivers only more confusion, but it can seem to be immensely sensible since it mimics the scientific approach. This leads to the horrific emergence of busy work in the place of real architectural work. Organizations experiencing many proof of concept projects that never make it to production is the most common symptom of this.

The way to combat complexity, real or perceived, is to allow our structural picture of the world to collapse and rebuild it a little differently. This requires a certain degree of humility. The post-structuralist rejection of essence often goes too far, and we must begin with an essence, even if we know it to be naïve, and allow repeated contemplation of a system to produce new concepts, to allow abduction to happen. In this, we believe in the convenient reductions just enough to allow them to be questioned and collapse. This is how the architect can use their relative a priori knowledge, but at the same time, employing fallibilism, doubt their experience, causing model collapse and reconstruction, which builds knowledge by discovering more and more attractors in the business context.

The key lesson of supervenience, of Bergson, of Broad, of James, is that essence and appearance can never be united, because they will necessarily drift from each other. Trying to keep the appearance still, or the essence perfect, leads to paralysis, to the software crisis, to a failure to understand how the world around us actually works. We cannot reject structure or essence, they are too central to the way we instinctively order the world, but we can adopt a humility that sees these structures and essences as fallible, fragile, and constantly breaking. Trying to hold emergent, supervenient systems still will always result in failure, but trying to avoid the challenge will also lead to catastrophe. The rejection of structure is necessary, but the refusal to engage with it, as often seen in the agile movement's rejection of architecture, leads to the same place.

Emergentism, strong emergentism, and supervenience explain the world around us, in the 21st century, in a much more coherent way

than the dogmas of empiricism, Platonism, or positivism. It is a simple theory with a great deal of explanatory power. The *élan vital* of Bergson is the supervenience, the constant driving of a system by itself, the unmoved mover of Aristotle. The process philosophies of Hegel, Deleuze, Bergson, and Whitehead allow us to appreciate this world and prepare for it. We can set aside arguments about whether emergence is a product of ignorance, or a deterministic world we can't understand. From a pragmatic perspective, we simply can't understand within the immediate timeframes of software projects, and so regardless of underlying ontology, we must act as if our world is emergent—from the perspective of our software, that's exactly what it is. Supervenience, when combined with the idea of attractors means that in many systems, we will see reoccurring states as the system moves around and also means that every system will eventually lurch somewhere we could not have imagined.

When working with emergent systems we must remember that it is still possible to engage with the world of things, of substance—in our computer code and in engineering constructs such as production lines. As engineers we are never tasked with building emergent systems—instead we build static systems that are at the mercy of supervenience which will push those static systems around. The tendency of software architecture thus far has been to try and make the emergent system static—this is what we instinctively have tried to hold still - and over the course of the next few walks we will reverse this. Instead of seeing the becoming as being, or being as becoming, we shall view our systems as being at the mercy of becoming.

Should the problem of determinism be solved before this gets to print, then you can ignore this book, but don't blame me, in a determinist world, it was already predestined that you would read it.

Complexity and Model Drift

Many of the philosophers we have met on throughout this book were struggling with the same problems that modern day software architects struggle with. Constantly attempting to hold one thing still, whilst the next generation would point out that when this was done, lots of other things would continue to move—giving rise to a Hegelian dialectic and constant churn of ideas and tools. Dijkstra’s mistake was not Dijkstra’s alone but was a feature of our thinking since Parmenides. The problem is one of complexity, that holding things still simply doesn’t work in emergent systems. Our abstractions are incredibly vulnerable, one weak snapshot in time of a system constantly moving. When we base our software designs on these abstractions, they in turn become incredibly vulnerable. The problem is one of model drift. As soon as we settle on a model of the world around us, the world starts to drift away from the model. The struggle between essence and appearance has very dramatic effects on the success of software architecture. In this view, Wittgenstein’s attempts to hold the world still with logic are what generates the nonsense he believes should be passed over in silence.

A Better Approach to Complexity.

To solve this problem, we must rise to the task of addressing the complexity that is obviously present in the social and business systems that require software. This is not the same, essential complexity across different kinds of system, or even the same kind of complexity across similar business systems. It is simply something that is beyond our control and comprehension.

Instead of a scientific, empirical investigation of complex systems, through probing and experimentation, it must be admitted that the concepts around every complex system being the same must be called into question. Complexity is a phenomenological concern and not something that is a universal property of everything we cannot understand—complexity is an attractor, not a universal.

When faced with a complex problem, that is, something which you do not fully understand, the approach needs to be one of concept creation—that is, philosophy by the definition of Deleuze. Empirical investigation is pointless if you are armed with a set of concepts that cause you to miss the right path through the system. As we have shown, the presence of the logical structuralism of the default architectural philosophy is a set of concepts that entrains the architect toward making the same mistakes over and over again in the belief that the complex social system can be engineered in the same way as any static system.

Software architecture is caught in a quandary. On the one hand, it is still about defining a rigid, logical structure that allows software functions to execute in a predictable way. On the other hand, it is about making decisions for how that code will be structured based on an emergent system, over which we have no control and no ability to predict. The software is the cold, hard, positivist model of an emergent system and as such is incredibly vulnerable as the emergent system shifts. Software architecture is therefore a problem unique in philosophy—it has immediate real-world effects

in the quality of software systems. On the one hand, the logic of the mathematically minded must prevail to ensure the system works, in the other, we must deal with multiplicity, the *élan vital*, supervenience, unpredictability, and flux. This has been the root of the software crisis, the reason that failure happens so often. Instead of learning the modes of thought presented by Whitehead, Bergson, Deleuze, or Heraclitus, the industry sought to bring the Platonic, the essentialist, to the Dionysian, to make hard science the master of the unknowable. It is one thing to believe that this will eventually be the case, but if I have a project deadline six months from now the chances of these problems being ironed out are quite small.

The question must then be raised, how does it ever work at all? There are after all millions of IT systems currently running to the point that our entire society depends on them.

The Deleuzian Walk

The Deleuzian walk describes more accurately than traditional frameworks how the architect approaches a problem in the real world. For Deleuze, we come to knowledge by repeatedly experiencing something, noting the differences each time. Remember that Deleuze believed that the only true essence that exists anywhere is difference—it's the one thing that is always present and is the fundamental building block of knowledge. This method is referred to as *transcendental empiricism*, by which Deleuze is interested in how we come to knowledge through real experience, as opposed to Kant's possible experience. Deleuze sees this process as passive—we don't even know we are doing it.

Deleuze describes our concepts as an emergent system. We passively integrate through repetition the differences, creating an emergent concept which in turn affects how we experience that which is directly experienced, which in turn influences the concept that is emergent. This is very different from Hegel's dialectic (as Deleuze was at great pains to point out), but it explains the constant movement in our concepts.

Architects will recognize this extremely pragmatic way of appreciating what they do. This idea enshrines the role of reflective practice, and seats it in the world of emergentism.

When we first take a walk in a new place, we will perhaps use a map. The map is an abstraction, a universal that applies to all walks in the world where a map exists. On our first walk, we are probably most concerned with not getting lost, and most of our attention is on the practical, turns, street names, directions. The next time we take the walk we will be open to noticing different things, and these differences define the walk for us, over time, as both we and the walk are changing. Perhaps we note the presence of mosquitos in the summer, dangerous corners for cars in the darkness of fall, slippery paths in the winter, beautiful views in the

summer. Walking with children will reveal differences we don't see from our perspective. Over time, the understanding of these many differences produces real knowledge of the walk, such that the individual taking the walk many times will build up knowledge far beyond the universal represented by the map. This idea is also discussed in Bergson's *Time and Free Will*. Walking across different seasons, with different people, at different times of day, in daylight and in darkness, exposes us to the attractors of the route, perhaps many times, each time noting the differences and building up knowledge.

The Deleuzian Walk, or Architectural Walking as it is called to make the idea more palatable in the IT industry, is the idea that the techniques, tools, and frameworks that an architect uses to move between the world of business flux and rigid software, is irrelevant. Every single tool exists simply to provoke a 'walk' around the flux of the business case. The more we walk, the greater the number of attractors we discover as we notice slight differences on every walk. Even tools and techniques that make the Platonic mistake of trying to find the essences of a problem serve a purpose in forcing us to walk from a different perspective. It is possible to use the wrong approaches, the wrong tools, the wrong thinking, and still arrive at a richer picture of what is actually happening. This explains why different teams approach problems with entirely different toolsets, sometimes with none, and can still deliver well. It is not the kinds of walks that matter, but the number, and the humility of the architect in knowing that the structures, diagrams, and models produced are superfluous to the actual understanding that is built up. This explains why a good architect can work within rigid confines of mandated tools and templates, or with no guidance at all, and still produce a good architecture. Architecture becomes a sense, a reflective practice. The tools and techniques of systems thinking also fall under this way of thinking, promoting a new walk from another perspective rather than saying anything scientifically grounded about a problem.

The concept of the Deleuzian Walk is an important one to understand. Once it is understood, it is possible to leave the constant attempts to argue that one tool, framework, or approach is better than the other, the problem of companies importing best practices and ideas from other companies without adjusting them for context disappears when we realize it is not the tool that is making the difference.

This is a powerful theory of architecture that goes a long way to explaining why the world of software design works the way it works, with obvious connections to Schön's reflective practice. Architecture as such is storytelling, based on imagination and intuition, but still requiring the technical ability to translate this to code structures.

The many tools and frameworks of IT architecture are simply performative, they do not solve real problems but facilitate this walking, and they provide a pseudo-scientific, analytic cover for gut feeling and imagination. The architectural tools we use are enshrined in the essentialist, structuralist world, and their combination allows architects to escape the limitations of this in the way Deleuze describes. The problem with this is that it takes many years for an architect to realize how to play this game. If we believe in these tools and frameworks, they form a tangled conceptual mess that actually gets in the way of interacting with the real world, they can choke and destroy the architect's ability to interact with the system—and such an architect will be accused of living in an ivory tower, detached from reality. Not all architects learn how to walk and remain stuck in pointless, navel gazing, overly intellectualized analysis, which leads nowhere and causes the rest of the organization to lose faith in architecture. The tools of architecture become powerful in the hands of the architect who doesn't actually believe in them, and a curse in the hands of those who do. Every architectural tool or framework encourages contemplation that causes the emergence of concepts, that enter into a supervenient loop, building the concept and changing it

constantly. Architecture can really happen when this loop starts to repeat in a way which we recognize—when we have experienced enough of the attractors in a system that differences cease to be shocking.

This book, as you’ve probably realized, is organized into a number of walks. Each walk is triggered by starting with a philosophical idea or thinker, forcing us to walk around the problems of software architecture, each time with a new lens, a new way of thinking, a new way of looking at the problems. Throughout the book we have been using ideas and thinkers to add new layers of knowledge to our philosophical understanding of architecture. Sometimes they contradict each other, leading to new walks, sometimes we need to repeat a walk with an old idea once we’ve discovered how to walk with a new one. Interestingly, in the reviewing of this book, philosophers found the structure unnerving and distracting, and architects found it natural.

The question now becomes: is there a better way to walk?

Walk 15: Residuality Theory

Now it is time to actually *do* some philosophy. Studying the philosophy of the past is interesting, but it's only useful if we use it to create new concepts with which to move forward. If we don't do this it is merely the history of thought, up to a present moment which is considered superior where all things are solved. There are thousands of books, courses, and lecture series describing the history of philosophy that go much deeper than this book. We have not embarked on this journey to merely describe our world but to change it. There was another famous philosopher who said something similar, but I can't remember which one¹.

Throughout this book I have been describing two worlds: the software, which is pure structure and ruled by the natural laws of science and logic, and the human environment that surrounds that software, which is best considered through an emergentist way of thinking. These two worlds are not separate. They are deeply connected. When the processes in the business environment warp and twist and turn around each other they destroy the fragile software architecture. Without software, the human system is critical, it is flexible and varied. With software, the entire system is brittle and inflexible, software brings with it a risk to the entire human system with its inability to keep up with changes in the environment. The two systems combine to form a dangerous chimera that we can't fully understand without being sure about when one perspective ends and the other begins.

It was to solve this final problem that I invented residuality theory².

¹It was Marx.

²O'Reilly, B. M. (2022). Residuality Theory, random simulation, and attractor networks. *Procedia Computer Science*, 201, 639-645.

The struggle that I have described in the previous Walks, that of certainty and uncertainty, of essence and appearance, of being and becoming, which has dominated Western thought for millennia, is brought to a point in the design of a software architecture, and the people we task to do this are only taught one perspective. This is the philosophical root of the software crisis.

Up to this point I have been describing the ideas of other thinkers and relating them to the practice of designing software applications. I have looked at the underlying philosophy that most developers and architects carry with them as they build out their systems and linked that philosophy to the frustrations we all have experienced working in this industry.

My conclusion has been that all architects who succeed in solving this problem engage in the Deleuzian walk, walking around the problem many times in different ways, allowing their conceptual model to collapse and be rebuilt. These walks are often inspired by different tools, frameworks, and methods. The tools and frameworks themselves, as I have argued, are not that important to the end result, but the act of walking is what adds to the architect's ability to build conceptual models that hold over time. This means that the journey to earn a good conceptual model involves a great deal of unnecessary overhead and ceremony.

Residuality theory solves this problem by making walking lightweight and fast. It starts by describing the entirety of the system, software and business context, as a *hyperliminal* system. A hyperliminal system is simply a system that consists of an ordered, structured, predictable part, and a disordered, emergent part. The emergent system exists before the structured part will be built, so this is specifically an engineering problem. To define residuality theory, we go back to Hume and state only that which we know to be true. No hypotheses, no speculation, no fanciful explanations or underlying belief system based on superstition. Our starting point will be one of emergentism and supervenience. These can be directly observed in any business system, so it is not

a matter of speculation but a matter of fact to state this. What can we say about this system? The only thing which we absolutely know to be true is that something in the system must happen, otherwise the system is dead and no longer interesting to us, or it has no emergent properties and is simple. (A dead or simple system is at equilibrium and a junior developer can cope with the task. Nothing will move; nothing will change). When something happens, the system must change in some way. The system will move to a new attractor. Whatever is left over of the software system, whatever still works, after this change we call the *residue*. In turn, this attractor is also an emergent system, and it will also change, creating yet another residue. This continues over time, never ending, with no real direction. Attractors and their residues are discovered through the use of *random simulation*, because we cannot determine them in any analytical, methodological way. The architect randomly creates *stressors*, things outside the current model that haven't been captured. These stressors are throwaway, existing only to open up discussions about potential attractors. There is no need for accuracy, only volume, as the method is designed to discover fault lines in the architecture along attractors that occur for multiple potential reasons. As we do this over and over again, we will start to notice that the system returns many times to the same attractors, until eventually it becomes very hard to discover new ones. This is a method of doubting, constantly doubting the attractors that our experience, our relative a priori, tells us to gravitate toward, always looking for more. This approach will cause our preconceived ideas to be questioned, preventing us from using products and trends to define the architecture by exploring attractors outside of those limited worldviews.

This method of doubting leads to a foundation, one where a software architecture is seen as a collection of residues, leftovers as the system moves from attractor to attractor. It can be viewed, at the beginning of its life, as a duration—Bergson's *durée*. It

is a multiplicity, indivisible, indeterminate, with many possible paths already present but not yet realized—what Deleuze would call *the virtual*. In an emergentist world, the software architect cannot control the system, cannot predict it, cannot even describe it, but the architect can control how residues are created in the software. In other words, the role of the software architect becomes determining how the software system will fall apart as the business system moves through its attractors. This is a very different view of architecture from that of the heroic builder, figuring everything out through analysis and arriving at a perfect design.

Residues are deliberately loose, there is no essence of a residue, it is simply whatever is left over, in the knowledge that whatever we describe will only be a partial truth. The residue becomes the arché, but a very weakly defined one, that allows us to reason about systems that move through the many trouser legs of a duration in the same way. This at once makes it possible to express an architecture across time and uncertain futures, freeing us from the standard, static, approaches of logical structuralism and traditional architecture. The residue provides a bridge between the deterministic, mechanistic software and the emergent business context, clearly demarcating the two things so that the architect doesn't get confused.

In this way the static architecture can have a number of static states which are accessible, loosely mapped to attractors in the business context. The business context is not reduced to a static number of snapshots, but through the use of attractors available to our imagination and intuition, we can produce an architecture of snapshots that is a reasonable reduction. The methods in residuality allow us to prove how effective this work has been. It is never possible to fully represent an emergent business context, and the residue serves as the abstraction to make sure that we don't fall into the trap of simplified static representations.

Once we have established these residues, building a software system becomes an exercise in making sure that in each residue,

as much of possible of the software survives. This means changing the original, naïve architecture so that this will actually happen, and managing the transition between residues. This allows the architecture to travel with the emergent system as it moves between attractors. This is shown to hold mathematically. Once this is done, the work of software engineering, creating the residues and testing them against expectations of how they will work, can be carried out without needing to think about the emergentism of the system—the work of the developer can be carried out inside the residue with a degree of certainty that is necessary to create components.

This is based on the model of an emergent system as a network of reoccurring attractors, and the method of random simulation relies on this. This can be empirically tested, as described in the first book, *before* implementation and this idea is easily shown to hold with simple experiments.^{3 4}

Residuality is thus a new *philosophy* of software architecture, embracing emergentism rather than essentialism and structuralism, and it works hard to avoid the pitfalls of these previously dominant, subconscious methods of investigating and defining software architectures in logical structuralism. It is at this philosophical level that the theory can be seen to be so wildly different from the ideas that have come before, not least because this theory is clear about its philosophical position from the start. Residuality is a philosophy of movement.

The use of ideas from emergentism, and from Bergson, Deleuze, and Hume, puts residuality in a process and post-structuralist vein of philosophy. Although it should be noted that it is easy to make many different arguments here—perhaps some will see the shadows of Descartes’ doubting and foundationalism in this, perhaps some will see a return to Platonism, with the residue as the essence or arché, others will see this as empiricism given the

³O'Reilly, B. M. (2022). Residuality Theory, random simulation, and attractor networks. *Procedia Computer Science*, 201, 639-645.

⁴O'Reilly, B.M. (2024)Residues: Time, Change and Uncertainty in software Architecture, LeanPub <https://leanpub.com/residuality>

focus on replication and experiment, others as Popperian given the substantial effort to refute the theory, others as pragmatism given the focus on the results in the real world. It is in any case a major departure from the previous ways of thinking about software architecture, in many cases because few people actually think about the philosophy behind their architectural position.

One thing to note here is the difference between residuality theory and traditional architectural design. In design, it is assumed that the designer is positively deciding the structure of the application in order to make it do what is needed. In residuality, the designer is naïve, making suggestions as to what might work, and then allowing the maelstrom of potential happenings to pull the design in new, unexpected directions. Instead of the still, static, architectural designs of the past, residuality can be thought of as Bambi learning to walk on ice, starting of with a naïve step and frantically readjusting until learning how to stay upright, learning at once the value of caution and different potentialities. In this world, ideas like low coupling (a universal) and high cohesion (an essentialist goal) are not something that the architect innately understands, plucks out of the air, or arrives at following a methodology, but instead something that emerges as a result of stressing the naïve conceptual model randomly. Thus the architect is freed from magically knowing how to do this. This suddenly makes it much easier to teach.

Many senior architects who have successfully grasped how to deliver in uncertainty have reported that the ideas of residuality seem obvious in hindsight. Residuality is nothing more than the same Deleuzian walking, but with faster, more efficient, more numerous walks devoid of ceremony, illusion, and logical structuralism.

Residuality then becomes a science of the emergent. Focused only on the manipulation of static software components, it aims to build applications that survive in emergent business contexts. There is no attempt to steer, change or control the emergent system. This is considered beyond us, it would require a new kind of science.

Architecture as Science

Now that I have arrived at a way of viewing the world and translated that to a way of acting in the world, it is time to ask questions of what has been created. Does it work? Popper claimed that the demarcation between science and pseudo-science was the ability of an idea to be falsified. The method behind residuality theory is testable, using an experiment to test for criticality (the ability to survive in unknown conditions) of the solution as the architect iterates through the random simulation. At this point, it moves from the philosophical underpinnings to an empirical experiment, and in the end, the journey through fashionable nonsense results in an empirically verifiable, scientific result, with a degree of rigor above and beyond the traditional approaches to software architecture.

What do we do with it?

The tools that are produced by this new philosophy are described in more detail in the first book. What philosophy makes possible is the creation of new tools, based on these concepts. The concepts steer our thinking and the design of the tools we need as we move into the future of software architecture with a newfound sense of who we are and what we are trying to achieve. We are no longer prisoners of the default philosophy of logical structuralism but instead have a better understanding of the problems we are trying to solve.

How should an architect think

Based on the journey so far, I am obviously calling for software architects to embrace emergentism. This view of the world allows us to embrace the certainty of software engineering, the logic, whilst still understanding that the world around our software is indeterminate, unpredictable, and, as yet, inaccessible to the world of science.

Walk 16: Conclusion.

The first question the overconfident software engineer will ask at this point of the book is “So what? What am I expected to do with this?” This is a fair question. I have, after all, drifted very far from the ideas of data structures and algorithms, relational databases and software patterns. For the extremely pragmatically minded, this book may not have been helpful at all. If this is the first question that occurs to you, then I should quiet your troubles by saying that absolutely nothing is expected of you. I hope you’ve enjoyed the journey across the history of Western thought and nodded a few times as you realized the parallels with the development of the ideas in software architecture.

For the rest of you, the point of this book is to free you from the invisible prison of the default philosophy of software architecture described in earlier walks, logical structuralism. Perhaps this book has come too late, and you have already realized many of these things across your career, but at least it will have given you a vocabulary to discuss these things with your colleagues across the industry. The idea behind this book is that if only we became aware of the default philosophy of software architecture that we could change it, that it would make it possible for us to develop new modes of thought, to question the sophistry of traditional software architecture, to develop new ideas based on new ways of looking at the world.

What I hope I have made clear is that there are many parallels between the world of philosophy and software architecture. Whilst we in tech may like to think that we are on the frontiers, pushing bold new ideas, the reality is that the history of Western philosophy has been there before us, often many times over. What should also be clear is the inherent aporia in breaking new ground. In

this book I have tried to separate the history of philosophy into separate movements, paradigms and thinkers, and I did the same with software architecture. The truth is that these things are much more entangled and inseparable than any narrative can convey, and when I separate the world of philosophy into thinkers and movements, I am also guilty of reductionism. The same applies when we separate the ideas of software architecture according to schools of thought, movements, or individual computer scientists. I do this only to create the narrative, to drive you to question your own ideas and your own thinking.

The journey of Western philosophy can be told as the relatively recent move from philosophies of stillness, through a period of doubt and questioning, to philosophies of movement, eventually leading to the new science of complex systems (although it is absolutely not neat and linear). The history of software engineering follows the same path, but at this present moment in time is reluctant to leave the safety of logical analysis and try something new—it is still stuck in its crisis of meaning.

The point of the book is to force you to pull your ideas apart, and then try to put them back together again, knowing that this can never result in the same set of ideas. To set you free from the clichés and myths and sophistry that run rife in our industry's collective thinking. In the previous Walk I outlined the ideas, methodologies, and techniques that I was able to come up with after rejecting the default philosophy, and my hope is that this book will start a movement of people doing the same thing, and that the next generation of software engineers interested in architecture will have a wealth and a diversity of ideas to choose from as they try to model the world around them.

Embracing the philosophies of movement will lead to new ways of thinking about software architecture, and residuality theory is a clear example of what can be created when the step away from the philosophies of stillness and logical structuralism is finally taken. This is the first step toward solving the architect's paradox.

In this book I have investigated the problem of the architect's paradox. That the logical approach of the computer programmer fails when trying to establish the conceptual integrity of a software application's structure. Much like in Russell's paradox, Turing's halting problem, and Gödel's incompleteness theorem, software's conceptual structure does not contain the means for its own validation, there must be something else. It is along this fault line that I now can define the job of the software architect, which is to know where the boundary lies between the static and the emergent, and how to work successfully on either side of it. Both of these tasks are deeply technical, and to do them well requires understanding of the huge philosophical differences in each problem, and the fact that trying to solve either side with the tools of the other inevitably leads to a software crisis.

In stating this, I have accidentally formulated a very clear definition of the job of the software architect. The ability to embrace this way of thinking, to see that the move from programmer to architect isn't simply about getting better at algorithms, or learning patterns, or learning MBA concepts, but about understanding the enormous philosophical differences between writing code and mapping real world structures to code. Architecture is therefore about developing this ability to produce software structures, and the first steps toward building this skill is to understand the philosophy that you carry with you, to the point where you can shape it and make it work for you. When you control the model, the model can't control you. And so, through the embrace of the philosophies of movement, I am finally able to answer Socrates' question "What is Architecture?" Architecture is decision making in the face of ignorance, the relating of the emergent to the static, the embrace of the world beyond logic and essence in order to design software structures for environments we don't yet, and may never, understand. To end the book with an answer, rather than aporia, is a triumph in itself.

Good luck in the library.

Acknowledgements

I'd like to thank my editor Dr Emily Herring who suffered bravely through the chaotic first drafts, my review team of Dr Fabien Imbault, Jeroen Haegebaert, David Whitney, Tanya O'Reilly, Dr Mahmoud Rasmi, Dr Riccardo Bennett-Lovsey, Blair Moir, Chris Simon, Glenn F Henriksen, Michele Frost, Shiraz Bhaiji, Russ Miles, Dr Einar Høst, Nick Travaglini, who all contributed with insights, proof reading, and vibes.

Cover art by Alexander O'Reilly, age 11. Layout by Hilde Goossens.